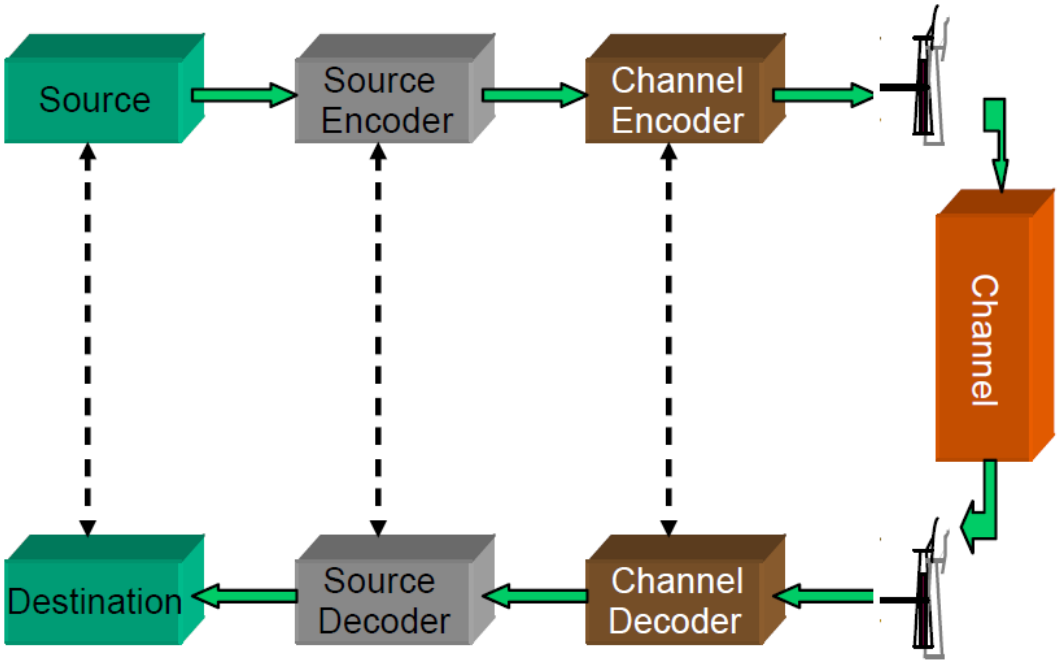
Digital communication principles

Digital communication systems



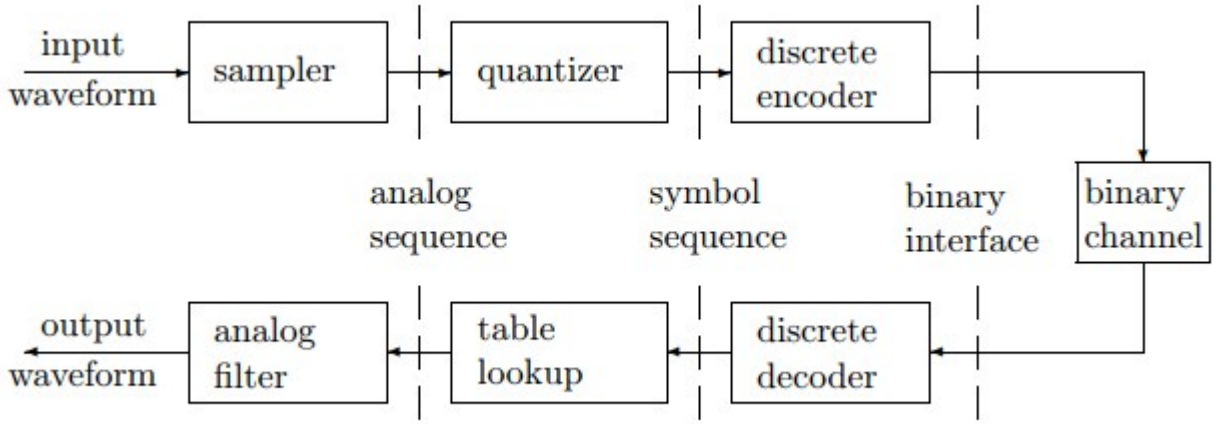
The source encoder

The source encoder

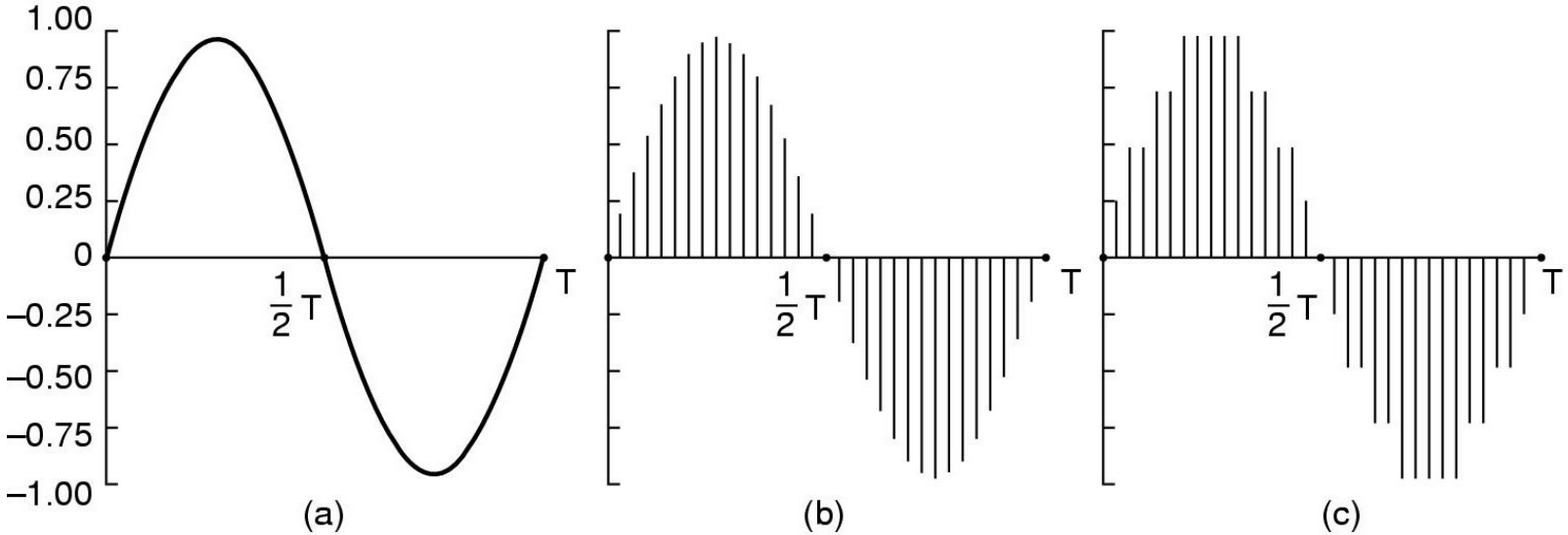
- Source encoder can be <u>discrete</u> (encoder's input is a sequence of symbols from a discrete alphabet like the latin alphabet; e.g. email message, web page) or <u>analog</u> (encoder's input is a waveform like a speech waveform; e.g. speech, video)
- Input: sequence of symbols or waveform
- Output: sequence of bits (which will be the input of the channel encoder/modulator)
- Goals of the encoder:
  - represent data as binary sequence
  - compress data

Detailed view of the source encoder

- discrete source encoders require only the inner layer above
- analog source encoders use all three layers



1)+2) Sampling & quantization



- a) the input waveform
- b) sampling the waveform
- c) quantizing the samples to 3 bits (i.e. 2^3 = 8 quantization levels)

1)+2)Sampling and quantization examples

- Telephone
  - 8.000 samples /sec (up to 4 KHz)
  - Needs 64 Kb/s (Pulse code modulation, PCM, 8-bit samples in Europe), or 56 Kb/s (USA, Japan – 7 bits)
  - Enhancements: Differential PCM, Adaptive DPCM
- Audio CDs
  - 44.100 samples /sec (up to 20 KHz)
  - 16-bit samples: quantization error is small but audible (the dynamic range of the ear is ca. 1 million)
  - Needs 705.6 Kb/s for mono, 1.411 Mb/s for stereo

3) Discrete Source encoding algorithms

- main goal: compression
- input: sequence of symbols from a finite alphabet (e.g. latin alphabet)
- output: sequence of binary digits
- must be uniquely decodable
- two types of encoding algorithms:
  - fixed code length algorithms: ASCII
  - variable code length algorithms: Huffman, LZW

Variable length codes for discrete sources

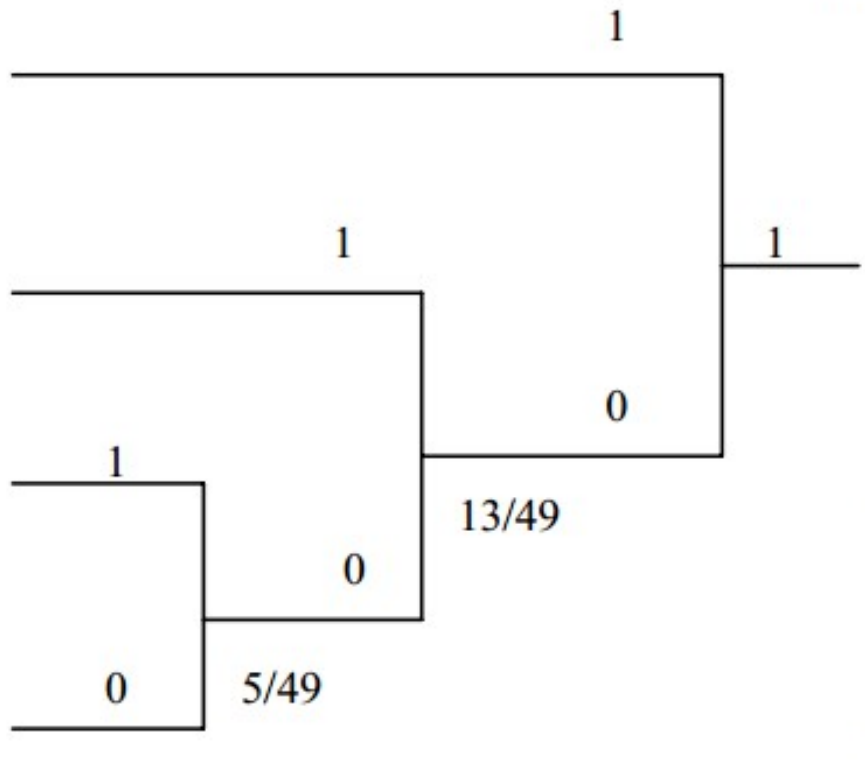
- A *variable-length code* C maps each source symbol a_j in a source alphabet $X = \{a_1, \ldots, a_M\}$ to a binary string $C(a_j)$, called a *codeword*. The number of bits in $C(a_j)$ is called the *length* $l(a_j)$ of $C(a_j)$.
- For example, a variable-length code for the alphabet $X = \{a, b, c\}$ and its lengths might be given by
  $C(a) = 0 \qquad l(a) = 1$
  $C(b) = 10 \qquad l(b) = 2$
  $C(c) = 11 \qquad l(c) = 2$

Variable length codes for discrete sources (1)

- A code C for a discrete source is *uniquely decodable* if, for any string of source symbols, say $x_1, x_2, \ldots, x_n$, the concatenation of the corresponding codewords, $C(x_1)C(x_2) \cdots C(x_n)$, differs from the concatenation of the codewords $C(x_1)C(x_2) \cdots C(x_m)$ for any other string $x_1, x_2, \ldots, x_m$ of source symbols.
- A *prefix* of a string $y1 \cdots yl$ is any initial substring $y1 \cdots ym$, $m \leq l$ of that string. The prefix is *proper* if $m < l$. A code is *prefix-free* if no codeword is a prefix of any other codeword. A *prefix-free code* is uniquely decodable.

Huffman encoding example

Symbol	Prob	Codeword	Length
" 2 "	36/49	" 1 "	1
" 3 "	8/49	" 01 "	2
" 1 "	4/49	" 001 "	3
" 0 "	1/49	" 000 "	3



$$l = \frac{36}{49}\cdot 1 + \frac{8}{49}\cdot 2 + \left(\frac{4}{49}+\frac{1}{49}\right)\cdot 3 = \frac{67}{49} = 1.4; \quad H = -\sum p_k \log p_k = 1.16.$$

Huffman encoding example (2)

- Huffman encode the sequence of symbols {3,2,2,0,1,1,2,3,2,2} using the codes from previous slide
- Code table:

Symbol	Codeword
0	000
1	001
2	1
3	01

- Coded sequence: {01,1,1,000,001,001,1,01,1,1}
  - Average bit rate: 18 bits/10=1.8 bits/symbol
  - Fixed length coding rate: 2 bits/symbol
  - Saving is more obvious for a longer sequence of symbol
- Decoding: table lookup

Huffman encoding algorithm

- **Step 1**: arrange the symbol probabilities in a decreasing order and consider them as leaf nodes of a tree
- **Step 2**: while there are more than one node:
  - Find the two nodes with the smallest probability and assign the one with the lowest probability a “0”, and the other one a “1” (or the other way, but be consistent)
  - Merge the two nodes to form a new node whose probability is the sum of the two merged nodes.
  - Go back to Step 1
- **Step 3**: For each symbol, determine its codeword by tracing the assigned bits from the corresponding leaf node to the top of the tree. The bit at the leaf node is the last bit of the codeword

More on Huffman encoding

- Huffman coding achieves the upper entropy bound
- One can code one symbol at a time (scalar coding) or a group of symbols at a time (vector coding)
- If the probability distribution is known and accurate, Huffman coding is very good (off from the entropy by 1 bit at most).

The LZW encoding algorithm

- The encoder and decoder dynamically build a dictionary of (symbol-code) pairs. This dictionary is initialized with the ASCII codes set (codes 1-255 are already occupied).
- The encoder always tries to add an unknown, long, symbol sequence to this dictionary and sends only the code for this sequence to the decoder (not the actual sequence)

The LZW encoding algorithm

```
Initialize dictionary with single character strings
P = first input character
WHILE not end of input stream
        C = next input character
        IF P + C is in the string table
                P = P + C
        ELSE
                output the code for P
                add P + C to the string table
                P = C
END WHILE
output code for P
```

The LZW decoding algorithm

```
Initialize dictionary with single character strings
OLD = first input code
output translation of OLD
WHILE not end of input stream
        NEW = next input code
        IF NEW is not in the string table
                S = translation of OLD
                S = S + C
        ELSE
                S = translation of NEW
        output S
        C = first character of S
        OLD + C to the string table
        OLD = NEW
END WHILE
```

LZW encoding example (1)

Input: ~~BA~~BAAB

Output: 66

P = ~~B~~ A

C = A

Dictionary:

255

256 BA

LZW encoding example (2)

Input: ~~BAB~~AAB

Output: 66, 65

P = ~~A~~ B

C = B

Dictionary:

255

256 BA

257 AB

LZW encoding example (3)

Input: ~~BABA~~AB

Output: 66, 65

P = ~~B~~ BA

C = A

Dictionary:

255

256 BA

257 AB

LZW encoding example (4)

Input: ~~BABAA~~B

Output: 66, 65,256

P = ~~BA~~ A

C = A

Dictionary:

255

256 BA

257 AB

258 BAA

LZW encoding example (5)

Input: ~~BABAAB~~

Output: 66, 65,256

P = ~~A~~ AB

C = B

Dictionary:

255

256 BA

257 AB

258 BAA

LZW encoding example (6)

Input: ~~BABAAB~~

Final Output: 66, 65,256,257

P = AB

C =

Dictionary:

255

256 BA

257 AB

258 BAA

LZW decoding example (1)

Input: ~~66, 65~~,256,257 Output:BA

Old= ~~66~~ 65

New=65

S=A

C=A

Dictionary:

255

256 BA

LZW decoding example (2)

Input: ~~66, 65,256~~,257 Output:BABA

Old= ~~65~~ 256

New=256

S=BA

C=B

Dictionary:

255

256 BA

257 AB

LZW decoding example (3)

Input: ~~66, 65,256,257~~

Final Output:BABAAB

Old= ~~256~~ 257

New=257

S=AB

C=A

Dictionary:

255

256 BA

257 AB

258 BAA

The channel encoder (Modulator)

- Has as input a binary sequence produced by the source encoder
- Produces a waveform signal suitable for being transmitted over a (wireless) channel

Channel capacity

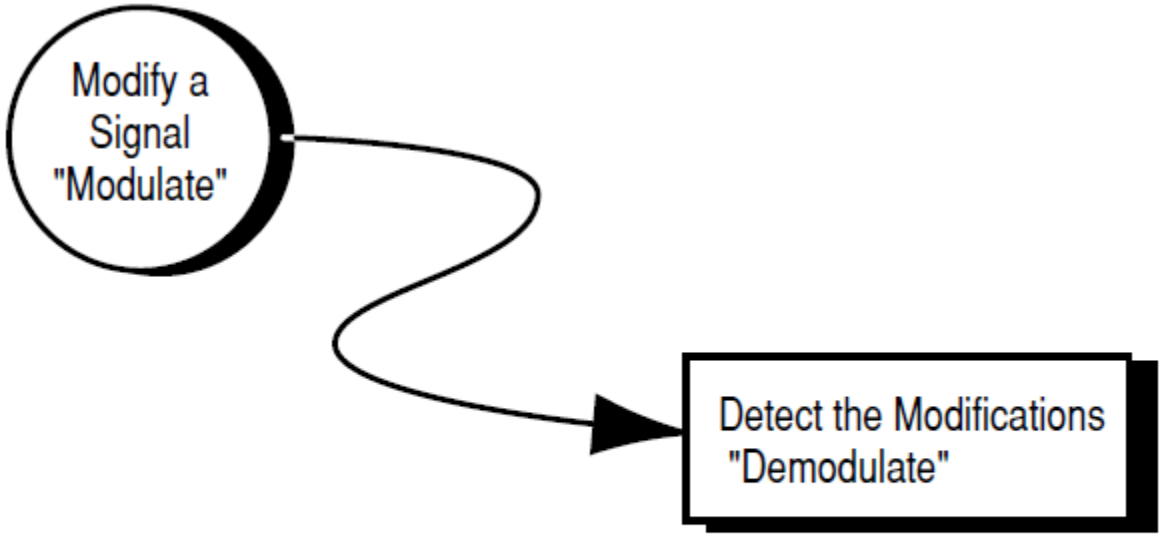
- Channel Capacity (C)
  - the maximum rate at which data can be transmitted over a given communication path, or channel, under given conditions
- Data rate (bps)
  - rate at which data can be communicated , impairments, such as noise, limit data rate that can be achieved
- Bandwidth (B)
  - the bandwidth of the transmitted signal as constrained by the transmitter and the nature of the transmission medium (Hertz)
- Noise (N)
  - impairments on the communications path
- Error rate - rate at which errors occur (BER)
  - Error = transmit 1 and receive 0; transmit 0 and receive 1

Reasons for choosing encoding techniques

- Digital data, digital signal
  - Equipment less complex and expensive than digital-to-analog modulation equipment
- Analog data, digital signal
  - Permits use of modern digital transmission and switching equipment
- Digital data, analog signal
  - Some transmission media will only propagate analog signals
  - E.g., unguided media (air)
- Analog data, analog signal
  - Analog data in electrical form can be transmitted easily and cheaply
  - E.g., AM Radio

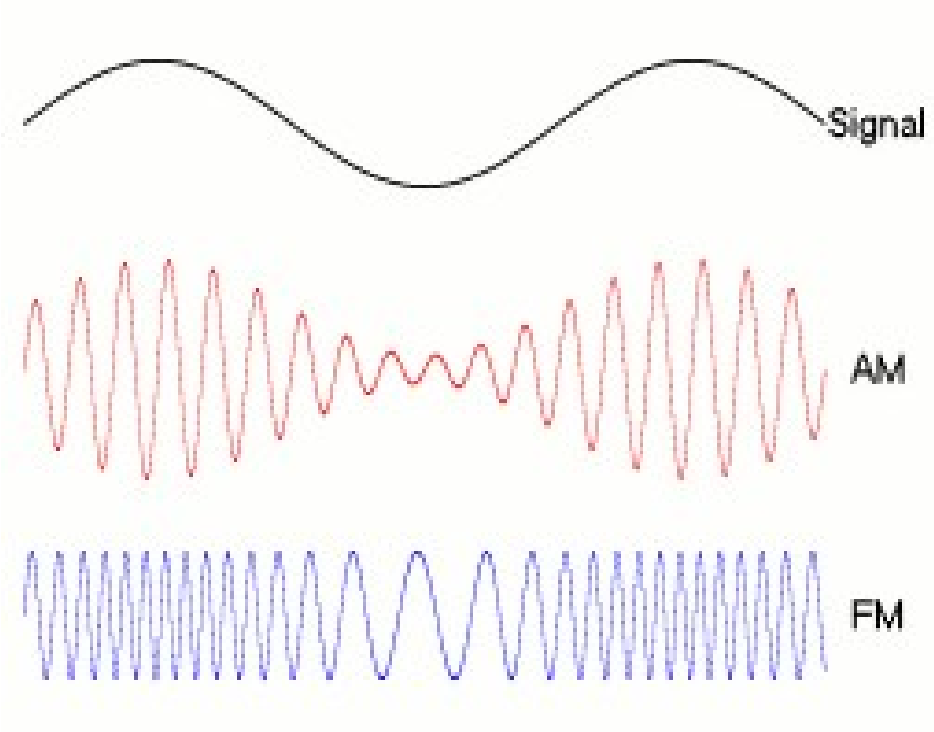
What is modulation ?

- Modulation = Adding information to a carrier signal
- The sine wave on which the characteristics of the information signal are modulated is called a carrier signal



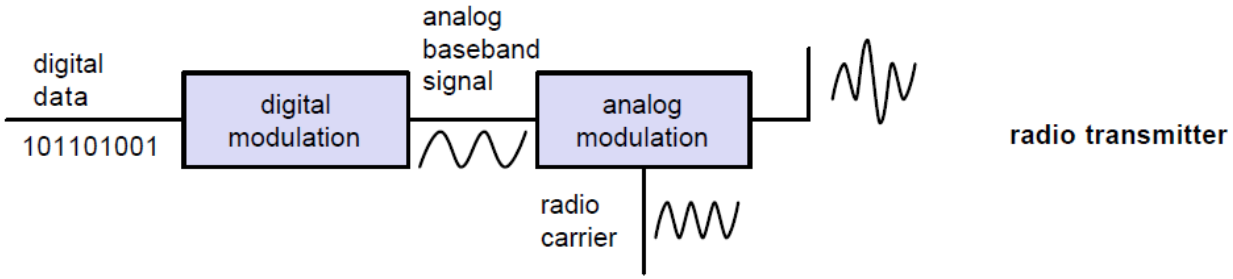
Any reliably detectable change in
signal characteristics can carry information

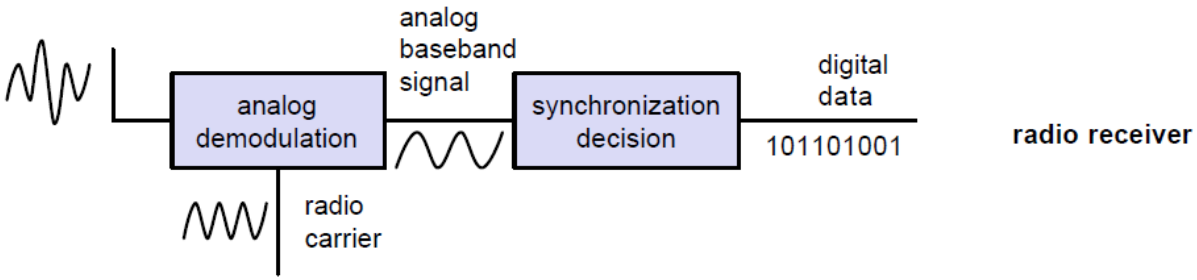
Analog modulation



Simple digital modulation and demodulation

Simple digital modulation and demodulation schemes





Signal characteristics that can be modified

signal $x(t) = A \cos(2\pi ft + \Phi)$

- A – amplitude
- f – frequency
- Φ – phase (initial angle of the sinusoidal function at its origin)

Modulation

- Modulation
  - Converting digital or analog information to a waveform suitable for transmission over a given medium
  - Involves varying some parameter of a carrier wave (sinusoidal waveform) at a given frequency as a function of the message signal
  - General sinusoid
    - $A \cos(2\pi f_C t + \varphi)$

Amplitude

Frequency

Phase

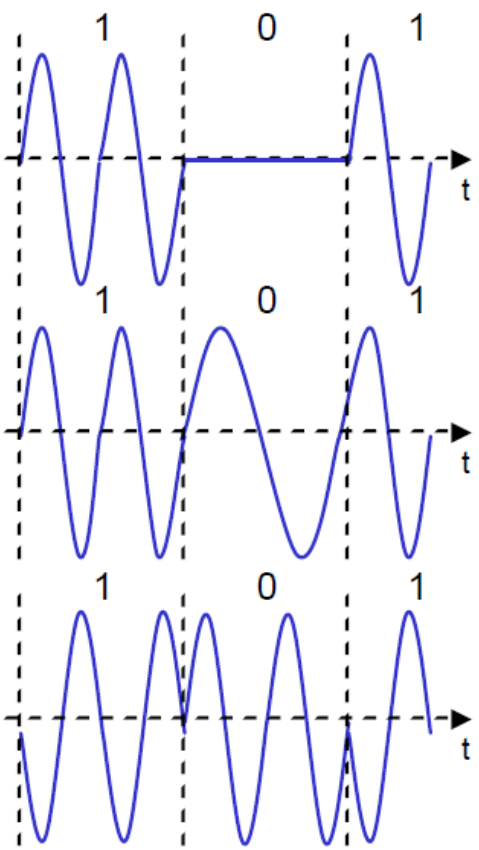
  - If the information is digital changing parameters is called "keying" (e.g. ASK, PSK, FSK)

Modulation (1)

- Motivation
  - Smaller antennas (e.g., λ /4 typical antenna size)
    - λ = wavelength = c/f , where c = speed of light, f= frequency.
    - 3000Hz baseband signal => 15 mile antenna, 900 MHz => 8 cm
  - Frequency Division Multiplexing – provides separation of signals
  - medium characteristics
  - Interference rejection
  - Simplifying circuitry
- Modulation
  - shifts center frequency of baseband signal up to the radio carrier
- Basic schemes
  - Amplitude Modulation (AM) — Amplitude Shift Keying (ASK)
  - Frequency Modulation (FM) — Frequency Shift Keying (FSK)
  - Phase Modulation (PM) — Phase Shift Keying (PSK)

Simple digital modulation techniques

- Amplitude Shift Keying (ASK):
  - change amplitude with each symbol – frequency constant
  - low bandwidth requirements
  - very susceptible to interference
- Frequency Shift Keying (FSK):
  - change frequency with each symbol
  - needs larger bandwidth
- Phase Shift Keying (PSK):
  - Change phase with each symbol
  - More complex
  - robust against interference

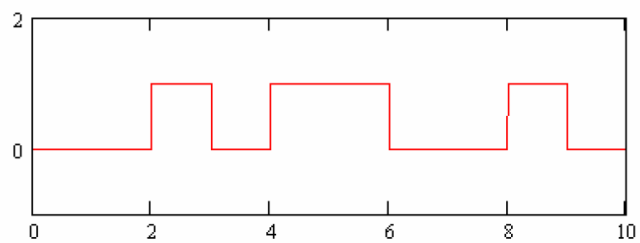


Simple digital modulation techniques

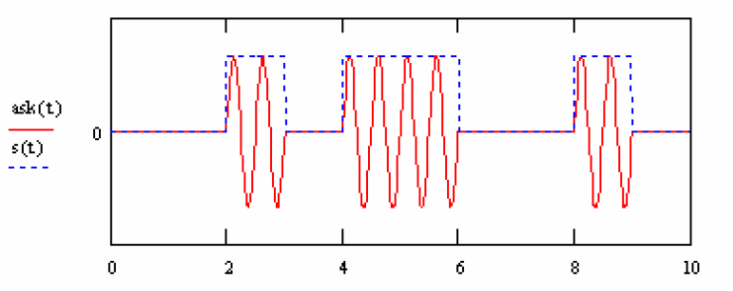
- In the following slides we briefly present binary ASK/ FSK/ PSK which are binary digital modulation schemes that encode each bit (0 or 1) into a separate waveform which is transmitted for a finite time interval T (i.e. the symbol duration interval)
- However these are rather inefficient, usually a block of b consecutive bits is mapped onto a symbol (also called a *signal*), all symbols generated are then transformed into a real waveform function which is then shifted up to the passband frequency band and transmitted (to be discussed in the complex modulation techniques slides following)

Binary ASK (Amplitude Shift Keying

- ASK On-off keying (Amplitude Shift Keying) – frequency is kept constant, amplitude has 2 levels (for bit 1 and for bit 0)



$$ASK(t) = s(t)\sin(2\pi f t)$$

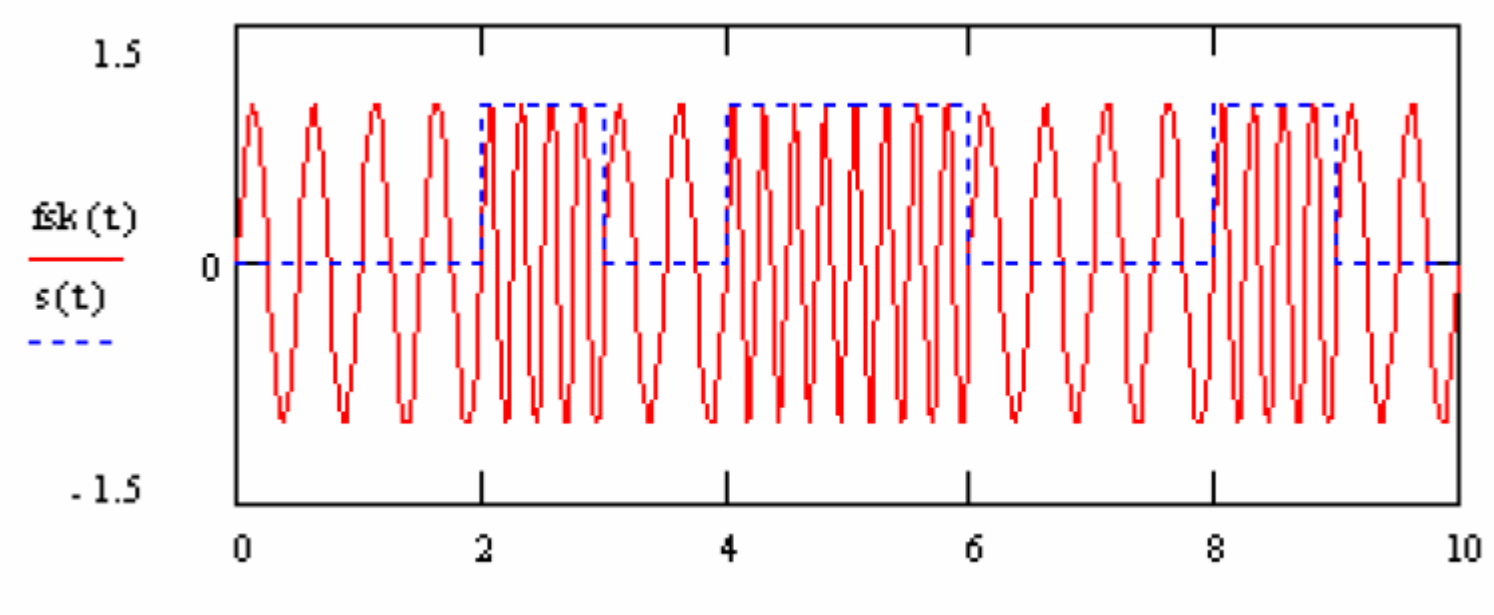


The binary sequence 0010110010

Binary FSK (Frequency shift keyring)

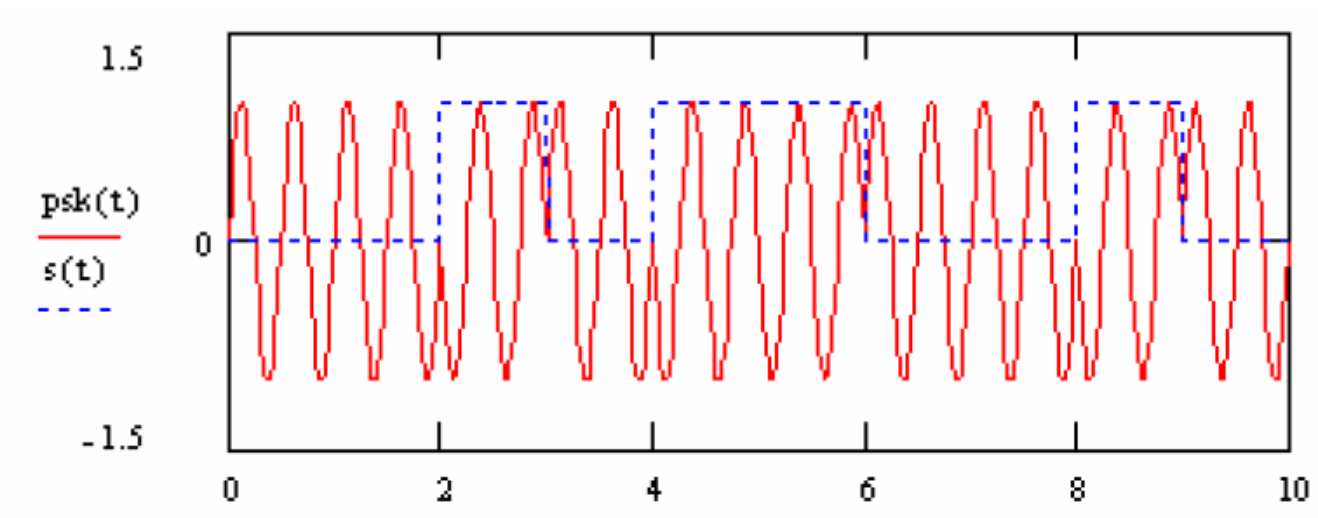
$$FSK(t) = \begin{cases} \sin(2\pi f_1 t) & for\ bit\ 1 \\ \sin(2\pi f_2 t) & for\ bit\ 0 \end{cases}$$

Instead of sin() functions, cos() functions can be used in the above formula.



Binary PSK (Phase shift keyring)

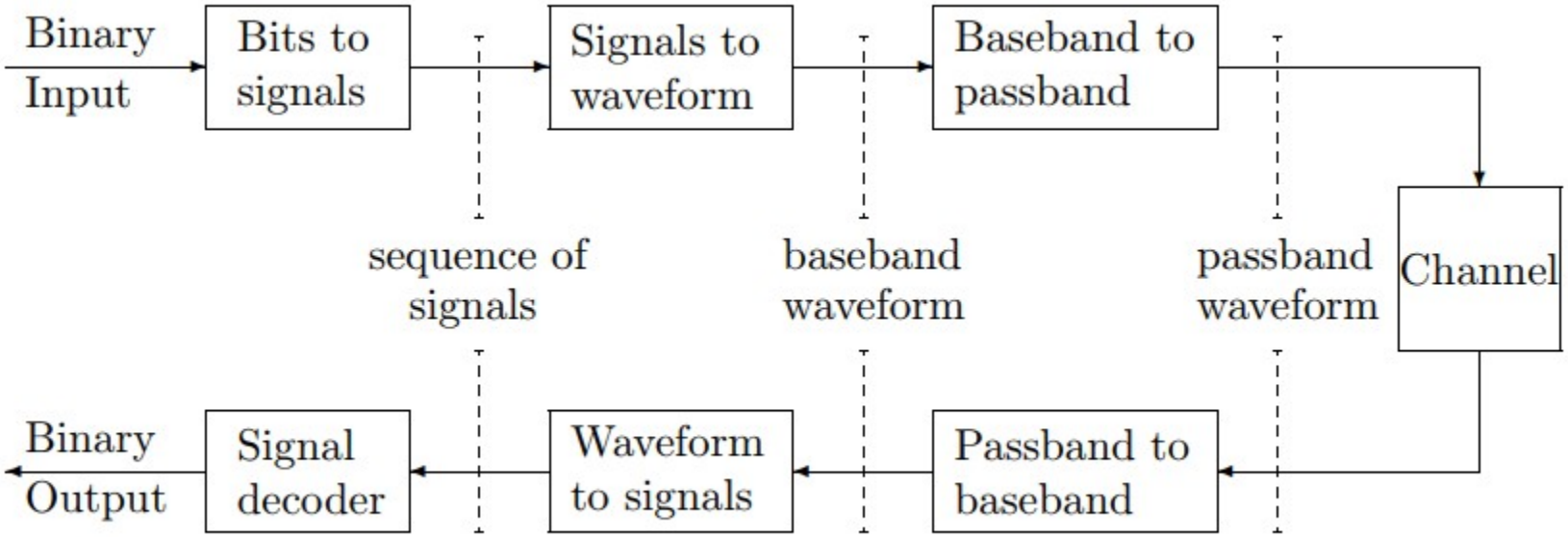
$$PSK(t) = \begin{cases} \sin(2\pi f t) & for\ bit\ 1 \\ \sin(2\pi f t + \pi) & for\ bit\ 0 \end{cases}$$



Instead of sin() functions, cos() functions can be used in the above formula.

Complex digital modulation and demodulation

Complex digital modulation and demodulation



Complex digital modulation (simplified)

1) The binary input (from the source encoder) is converted to <u>signals</u> (i.e. vectors of real/complex numbers)

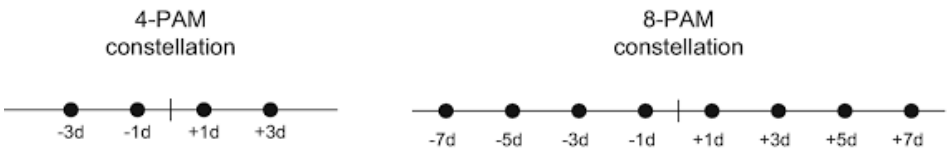
2) The signals are transformed/expanded into a waveform – <u>continuous function of time</u> (this waveform is computed using an orthonormal/orthogonal expansion, e.g. using values from the signals vector as coefficients of a Fourier Series waveform (computed using the Discrete Fourier Transform))

3) The resulted baseband waveform (i.e. centered around zero frequency) is then transformed into a passband waveform (i.e. a waveform centered around a higher frequency due to national and international regulations) – <u>continuous function of time</u> .

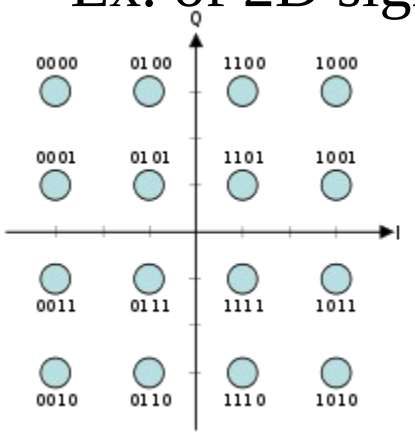
Step 1. Bits to signal

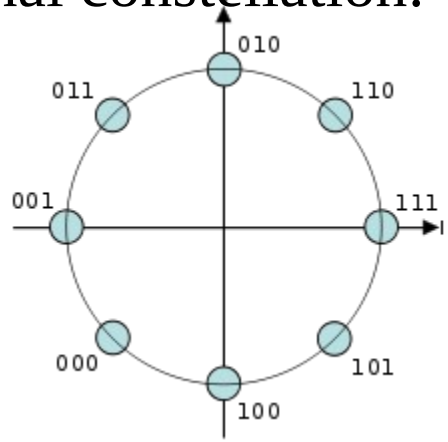
1) Incoming bits are segmented into b-bit blocks
2) b-bit blocks are mapped to a <u>signal constellation</u> A = $\{a_1, a_2, \ldots a_M\}$ where $M=2^b$ possible blocks; each value of the signal constellation is called a <u>signal</u>(it is just a real/complex number)

Ex. of 1D signal constellation



Ex. of 2D signal constellation:





Step 2. Signals to waveform

The obtained sequence of signals u_1, u_2, u_3, … is then mapped to a waveform u(t) (i.e. a continuous time function) using an orthogonal/orthonormal expansion p(t):

where T is the interval between successive signals.

Fourier series are an example of orthonormal/orthogonal expansion and are summarized in the following slides.

Fourier Series

- The simplest way of representing a waveform (continuous function of time) as a vector of values (i.e. Fourier coefficients)
- Fourier series represent a real/complex waveform (i.e. oscillating function of time) as a weighted sum of sinusoids (sin() and cos() functions). Each weight (i.e. Fourier coefficient) is determined by the function and the function expression is determined by the set of weights (<u>so a continuous math function can be approximated by a computer vector of coefficients</u>).
- Let a continuous function of time, integrable and periodic on the interval [-T/2, T/2] with period T
- The Fourier series of function u(t) is:

  for t in [-T/2,T/2] and u(t)=0 elsewhere where the Fourier coefficients are:

Fourier Transform

- Fourier transform maps a function of time into a function of frequency . The Inverse Fourier transform maps back to .

- There are theorems (Plancherel) that say that given any function integrable u(t), there exist its Fourier transform, ; Also, given the Fourier transform , there exist the function u(t).

Fourier Transform (cont.)

- Although the Fourier transform uses an infinity of Fourier coefficients, there are theorems that prove the function u(t) is also approximated by a finite number of Fourier coefficients (, , …)
- Euler’s formula
- Using Fourier transform, we can approximate a waveform as a vector of real numbers and having a vector of samples (real numbers) we can approximate/interpolate a waveform (continuous, oscillating function)

Step 3. Baseband to passband

- The baseband waveform is shifted up to a passband waveform (by increasing its frequency to be around a central carrier frequency f_c).
- The baseband waveform *u(t)* is made of frequency components centered at 0 frequency; i.e. its Fourier transform is zero except for the frequencies
- The passband waveform *x(t)* will be made of frequency components centered around the carrier frequency f_c; i.e. its Fourier transform is zero except for
- The baseband waveform *u(t)* is shifted up to frequency f_c by multiplying it with . Depending if *u(t)* is real or complex, it may be necessary to apply additional transformation in order to get to a real (not complex) passband waveform.

Example of digital modulation techniques

1. Simple example of binary modulation algorithm

- A sequence of binary symbols (bits) enters the modulator at T-spaced instants of time
- These symbols can be mapped into real numbers using the mapping: 0 +1 and 1 1
- The resulted sequence of real numbers u_1, u_2, u_3, … is then mapped into a baseband waveform given by:

where

- Then, this baseband waveform is transformed into a passband waveform centered around a carrier frequency
- In the absence of noise, the receiver can sample u(t) at times T, 2T, 3T, … to retrieve u_1, u_2, u_3, … which can then be decoded into the original binary symbols

Simple example of binary modulation algorithm (part 2)

- each pulse waveform component is time limited (i.e. is transmitted on a limited period of time) to T (i.e. the duration of a symbol)
- the receiver (in the absence of noise) just samples the received signal at times T, 2T, 3T, … in order to retrieve the sent signals u_1, u_2, u_3, … ; remember that is 1 when x=0 and is zero for the other integer values of x (i.e. it is ideal Nyquist).

…..

2. PAM (Pulse Amplitude Modulation)

1) Incoming bits are segmented into b-bit blocks
2) b-bit blocks are mapped to a signal constellation $A = \{a_1, a_2, \ldots a_M\}$ where $M=2^b$ possible blocks; each value of the signal constellation is called a <u>signal</u>(it is just a real/complex number)
3) The obtained sequence of signals $u_1, u_2, u_3, \ldots$ is then mapped to a waveform u(t) (i.e. a continuous time function) by the use of time shifts of a basic pulse waveform p(t):

   where T is the interval between successive signals
4) Baseband waveform *u(t)* is transformed to passband waveform

1D signal constellation example for PAM

- for 2 signals (bit blocks: 0 and 1):

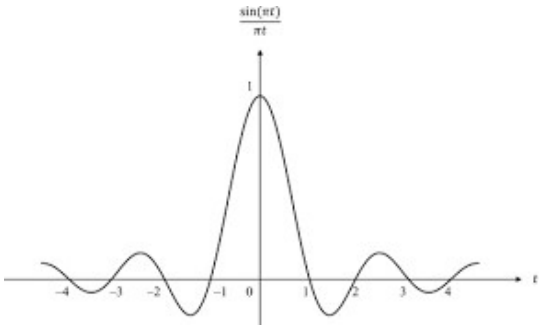
  -1, 1

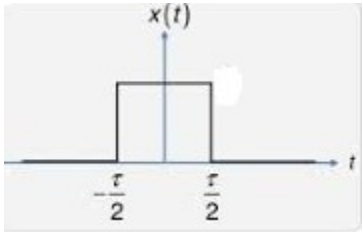
- for 4 signals (bit blocks: 00, 01, 10 and 11):

  -3,-1,1,3

- for 8 signals (bit blocks: 000, 001, 010, 011, 100, 101, 110, 111): -7,-5,-3-1,1,3,5,7
- the distance between 2 signals in a signal constellation does not necessary need to be 2, but it should be a constant; for example, this is also a signal constellation with 4 signals: -9,-3,3,9

p(t) function example for PAM





- raised cosine function
- Gaussian basis pulse

PAM passband transform

- in order to shift the *u(t)* signal waveform to a higher frequency band, the signal is multiplied with , which makes the Fourier transform zero except for where B is the frequency bandwidth of the original signal
- resulted waveform is now a complex which can not be transmitted (only real waveforms can be transmitted)
- so, *u(t)* is also multiplied with the complex conjugate of , i.e. , so the transmitted signal becomes:

which is a real signal centered at frequency f_c

3. QAM (Quadrature Amplitude Modulation)

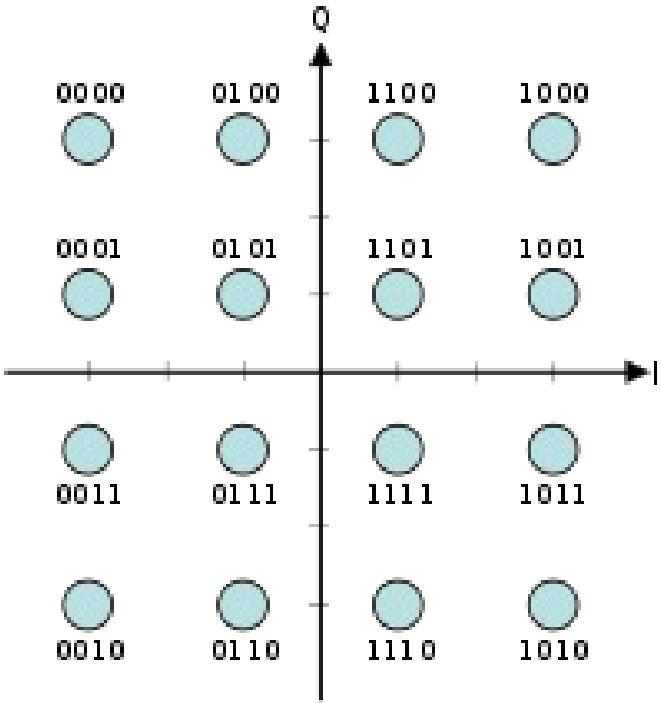
1) Incoming bits are segmented into b-bit blocks
2) b-bit blocks are mapped to a signal constellation $A = \{a_1, a_2, \ldots a_M\}$ where $M=2^b$ possible blocks; a_i is a complex number and $A = \{(a+bi) \mid a \in \mathfrak{R}, b \in \mathfrak{R}\}$; each signal can be considered a tuple of two real numbers
3) The obtained sequence of signals (i.e. complex numbers) u_1, u_2, u_3, … is then mapped to a waveform u(t) (i.e. a continuous time function) by the use of time shifts of a basic pulse waveform p(t) (the same as for PAM, ideal Nyquist):

   where T is the interval between successive signals

4) Baseband waveform *u(t)* is transformed to passband waveform

Constellation example for 16-QAM

- One axis represents the in-phase component and the other represents the quadrature component



QAM baseband to passband

- in order to shift the *u(t)* signal waveform to a higher frequency band, the signal is multiplied with , which makes the Fourier transform zero except for where B is the frequency bandwidth of the original signal
- resulted waveform is complex which can not be transmitted (only real waveforms can be transmitted)
- so, the complex conjugate is added to , so the transmitted signal becomes:

which is a real signal centered at frequency f_c

Re{x} is the real part of complex number *x* and *Im{x}* is the imaginary part of complex number *x*

QAM decoder (part 1)

- in the absence of noise, the received signal is

where I(t) is the in-phase component and the Q(t) is the quadrature component; they are our of phase by 90° -> orthogonal

- The received signal *r(t)* is multiplied separately by and by so we get:

and respectively

QAM decoder (part 2)

We have:

The high frequency terms (containing) can be eliminated using a low-pass filter, leaving only the in-phase component *I(t)*.

Similarly for the quadrature component:

Similarly, the high frequency terms (containing) can be eliminated using a low-pass filter, leaving only the in-phase component *Q(t)*.

HUFFMAN ENCODING EXAMPLE

Text = a b a a c b b d a c
P(a) = 4/10 = 0.4
P(b) = 3/10 = 0.3
P(c) = 0.2
P(d) = 0.1

```
                    1
                   / \
                0/    \1
               /       \
0=C(a)<=0.4    0.6
                     / \
                  0/   \1
                  /     \
   10=C(b)<=0.3  0.3
                       / \
                    0/   \1
                    /     \
     110=C(c)<=0.2   0.1=>C(d)=111
```

Avem urmatoarele coduri Huffman:
C(a) = 0
C(b) = 10
C(c) = 110
C(d) = 111

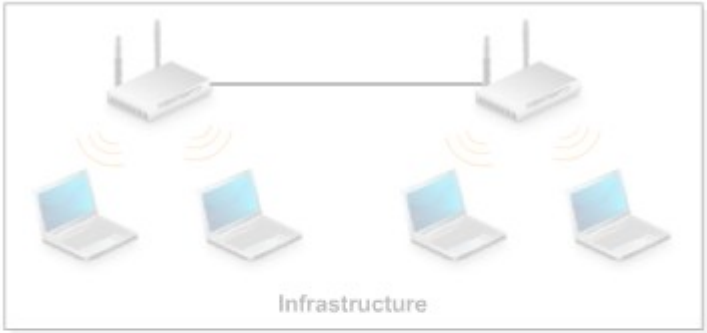
Length[C(Text)] = 1*4 + 2*3 + 3*2 + 3*1 = 19 biti ~ 1.9 biti per simbol

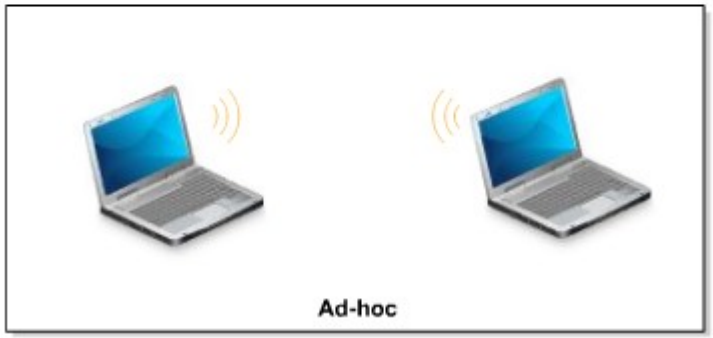
Codificare de lungime fixa: 2 biti pt fiecare simbol
Length[Cfix(Text)] = 10*2 = 20 biti ~ 2 biti per simbol

IEEE 802.11 Wireless Local Area Networks (Wi-Fi-LANs)

Types of Wireless LANs

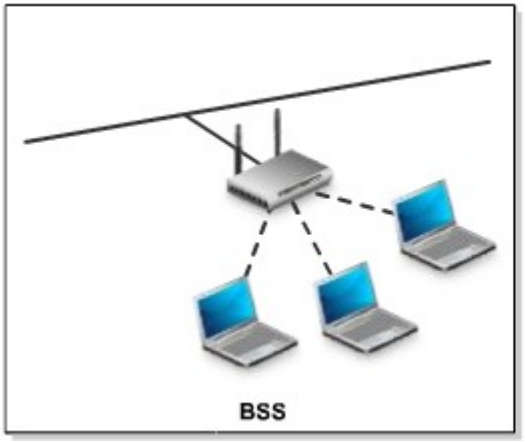
- Infrastructure (BSS and ESS)
- Ad-hoc (BSS)

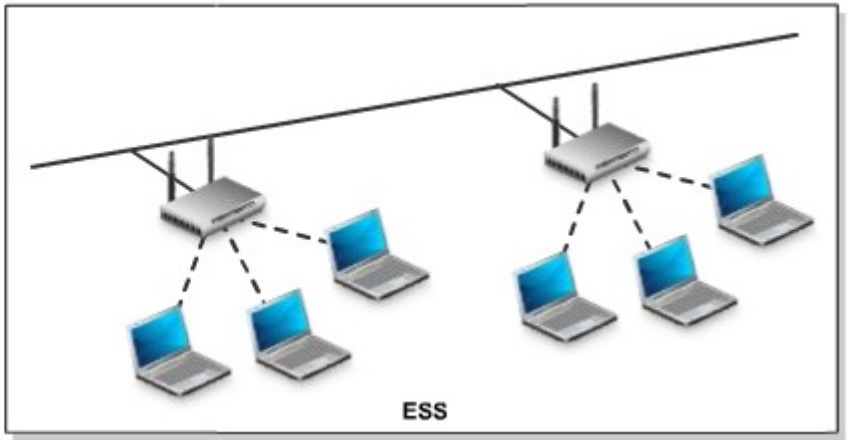




Wireless network implementation

- SSID – 32 long alfanumeric string identifying the WLAN
- BSS (Basic Service Set) – a network consisting of several clients and a wireless Access Point (AP); unique SSID
- ESS (Extended Service Set) – a network consisting of several wireless AP; adds mobility, Aps can use different SSIDs





IEEE 802 LAN standards and TCP/IP model

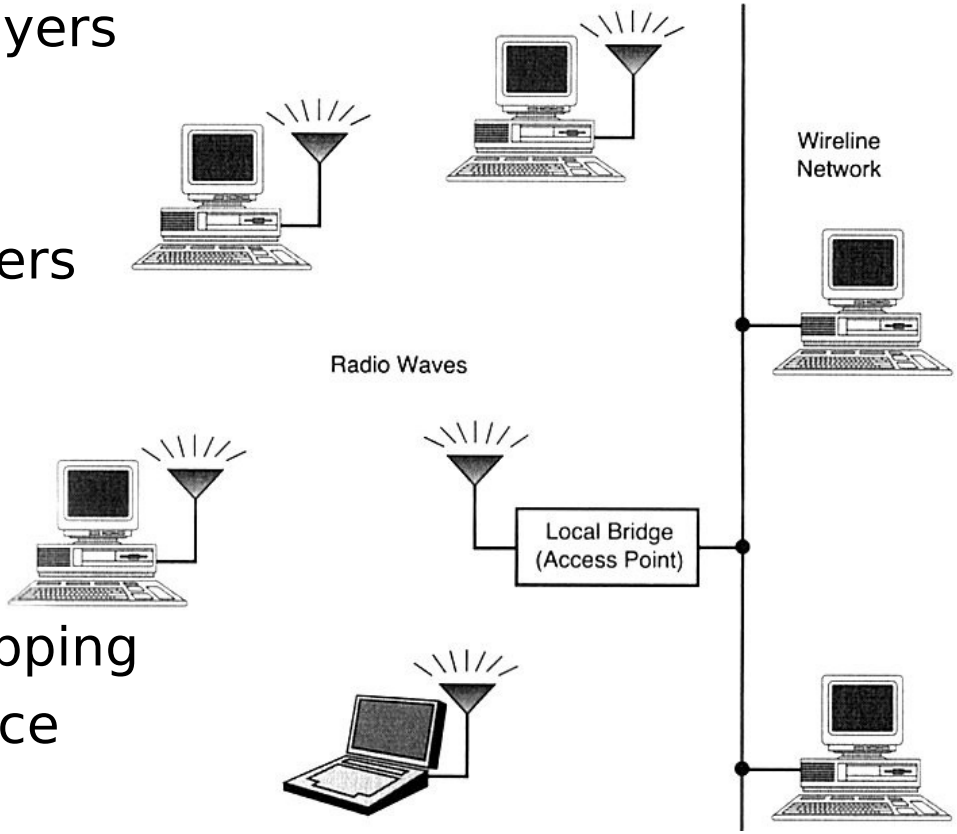
- The IEEE 802.x LAN standards deal with the DataLink and Physical layer of the TCP/IP model

Wireless network concepts

- Wireless host – computer, smartphone etc.
- Wireless link – the connection between hosts and base stations over atmospheric space
- Base station – responsible for sending/receiving data between wireless hosts and the larger (wired) network; coordinates transmission of multiple transmission hosts; ex. IEEE 801.11 access points and cellular Base Transceiver Station

802.11 WLANs - Outline

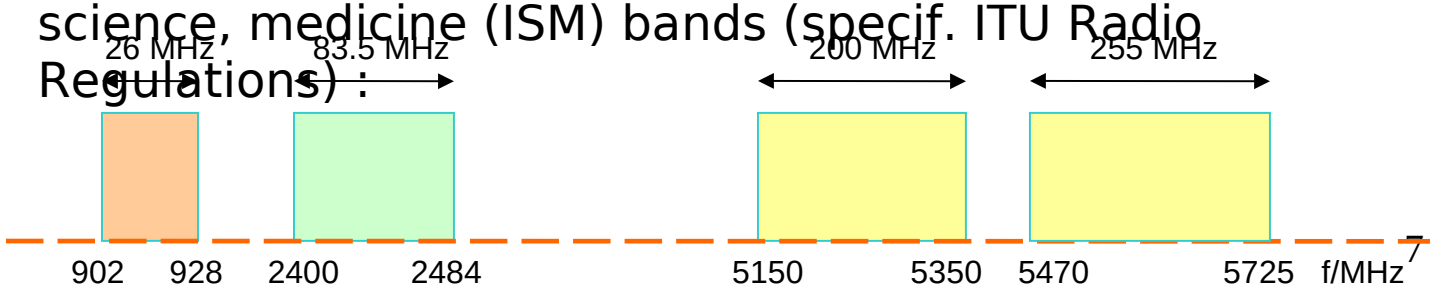
- 802.11 bands and layers
- Datalink layer
- Media access layer
  - frames and headers
  - CSMA/CA
- Physical layer
  - frames
  - modulation
    - Frequency hopping
    - Direct sequence
    - Infrared
- Security
- Implementation



Based on: Jim Geier: Wireless LANs, SAMS publishing and IEEE 802 - standards

802.11 WLAN technologies

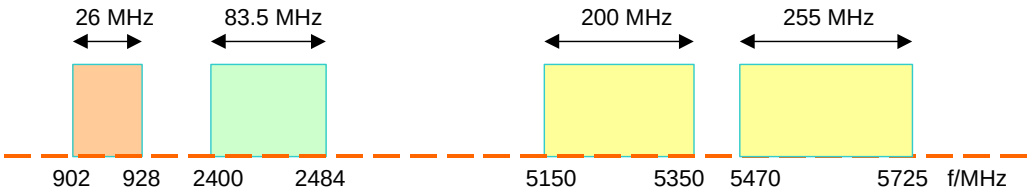
- IEEE 802.11 standards and rates
  - IEEE 802.11 (1997) 1 Mbps and 2 Mbps (2.4 GHz band)
  - IEEE 802.11b (1999) 11 Mbps (2.4 GHz band) = Wi-Fi
  - IEEE 802.11a (1999) 6, 9, 12, 18, 24, 36, 48, 54 Mbps (5 GHz band)
  - IEEE 802.11g (2001 ... 2003) up to 54 Mbps (2.4 GHz) backward compatible to 802.11b
  - IEEE 802.11n (2009) Multiple-Input-Multiple-Output (MIMO) antennas, 2.4GHz and 5GHz bands, 54-600Mbps
  - IEEE 802.11ac (2013-2015), MIMO, more spatial streams, wider channels, up to 1300 Mbps .... IEEE 802.11ax (future)
- IEEE 802.11 networks work on license free industrial, science, medicine (ISM) bands (specif. ITU Radio Regulations) :



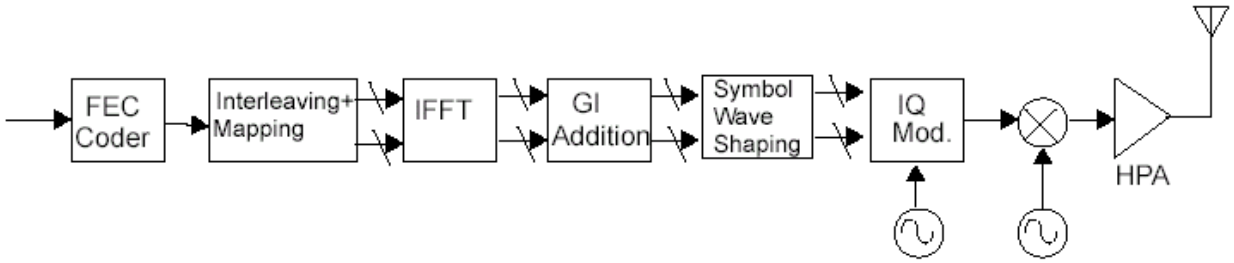
Other WLAN technologies

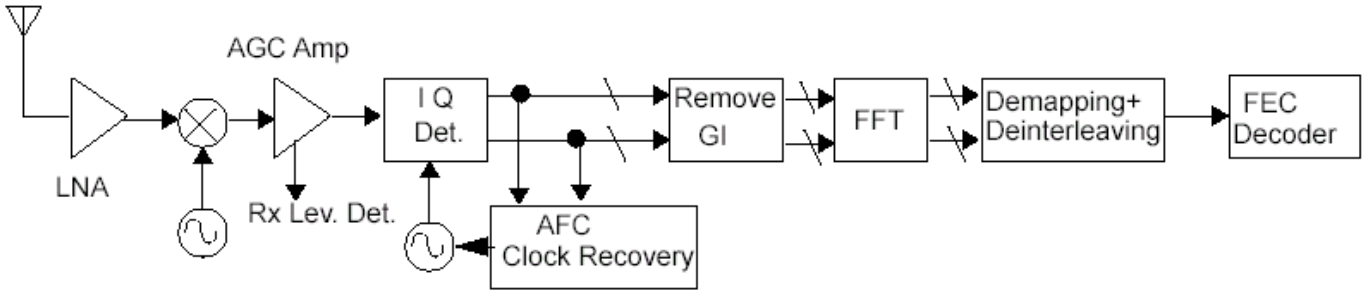
- High performance LAN or HiperLAN (ETSI-BRAN EN 300 652) in the 5 GHz ISM
  - version 1 up to 24 Mbps
  - version 2 up to 54 Mbps
- HiperLAN provides also QoS for data, video, voice and images
- Bluetooth
  - range up to 100 meters only (*cable replacement tech*.)
  - Bluetooth Special Interest Group (SIG)
  - Operates at max of 740 kbps at 2.4 GHz ISM band
  - Applies fast frequency hopping 1600 hops/second
  - Can have serious interference with 802.11 2.4 GHz range network

802.11a



- Operates at 5 GHz band
- Supports multi-rate 6 Mbps, 9 Mbps,… up to 54 Mbps
- Use Orthogonal Frequency Division Multiplexing (OFDM) with 52 subcarriers, 4 us symbols (0.8 us guard interval)
- Use inverse discrete Fourier transform (IFFT) to combine multi-carrier signals to single time domain symbol



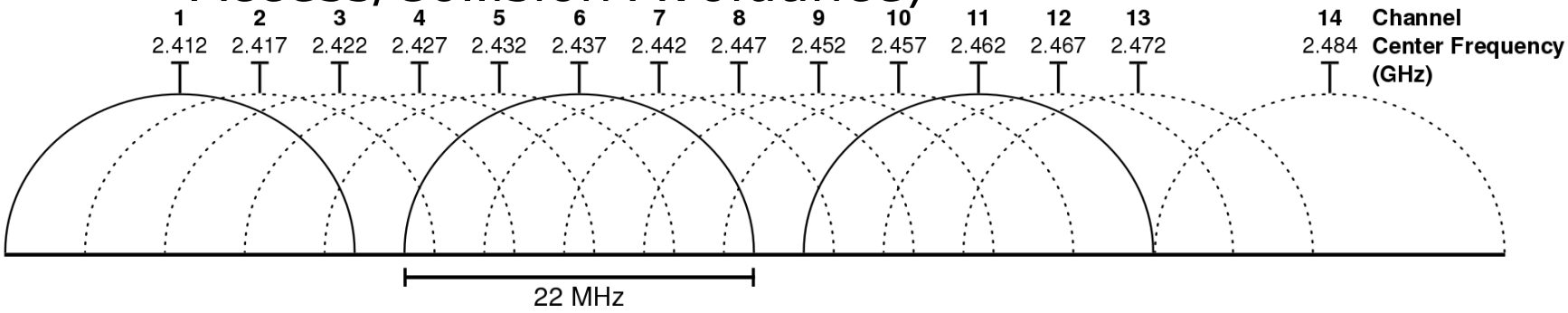


IEEE 802.11a rates and modulation formats

Data Rate (Mbps)	Modulation	Coding Rate	Coded bits per sub-carrier	Code bits per OFDM symbol	Data bits per OFDM symbol
6	BPSK	1 / 2	1	48	24
9	BPSK	3 / 4	1	48	36
12	QPSK	1 / 2	2	96	48
18	QPSK	3 / 4	2	96	72
24	16QAM	1 / 2	4	192	96
36	16QAM	3 / 4	4	192	144
48	64QAM	2 / 3	6	288	192
54	64QAM	3 / 4	6	288	216

802.11b

- obtains a max raw throughput of 11Mbps
- uses 2.4GHz band
- uses DSSS (Direct-sequence spread spectrum) modulation
- uses CSMA/CA (Carrier Sense Multiple Access/Collision Avoidance)

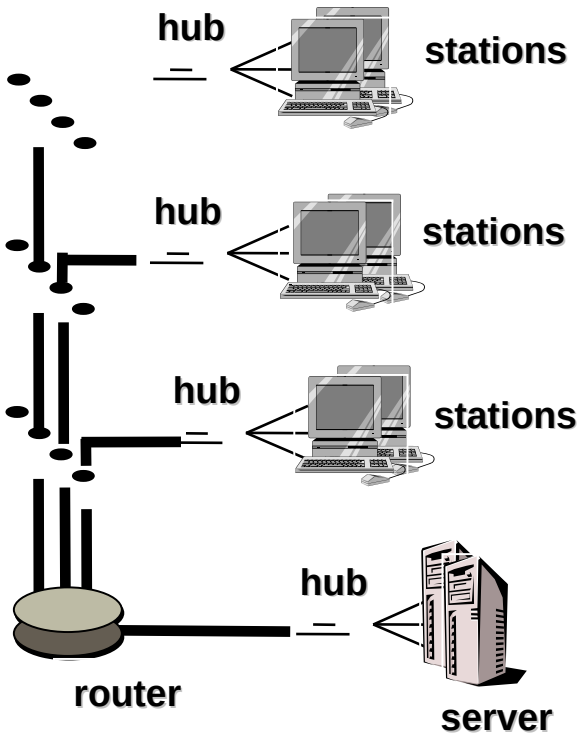


802.11g

- obtains a max raw throughput of 54Mbps
- uses 2.4GHz band
- uses OFDM (Orthogonal Frequency Division Multiplexing) modulation; has 52 OFDM subcarriers with a carrier separation of 0.3125 MHz
- uses CSMA/CA (Carrier Sense Multiple Access/Collision Avoidance)
- backward compatible with 802.11b (same number of channels)

IEEE 802-series of LAN standards

- 802 standards free to download from http://standards.ieee.org/getieee802/portfolio.htm



IEEE 802®: Overview & Architecture

IEEE 802.1 ™ Bridging & Management

IEEE 802.2 ™: Logical Link Control

IEEE 802.3 ™: CSMA/CD Access Method

IEEE 802.4 ™: Token-Passing Bus Access Method

IEEE 802.5 ™: Token Ring Access Method

IEEE 802.6 ™: DQDB Access Method

IEEE 802.7 ™: Broadband LAN

IEEE 802.10 ™: Security

IEEE 802.11 ™: Wireless

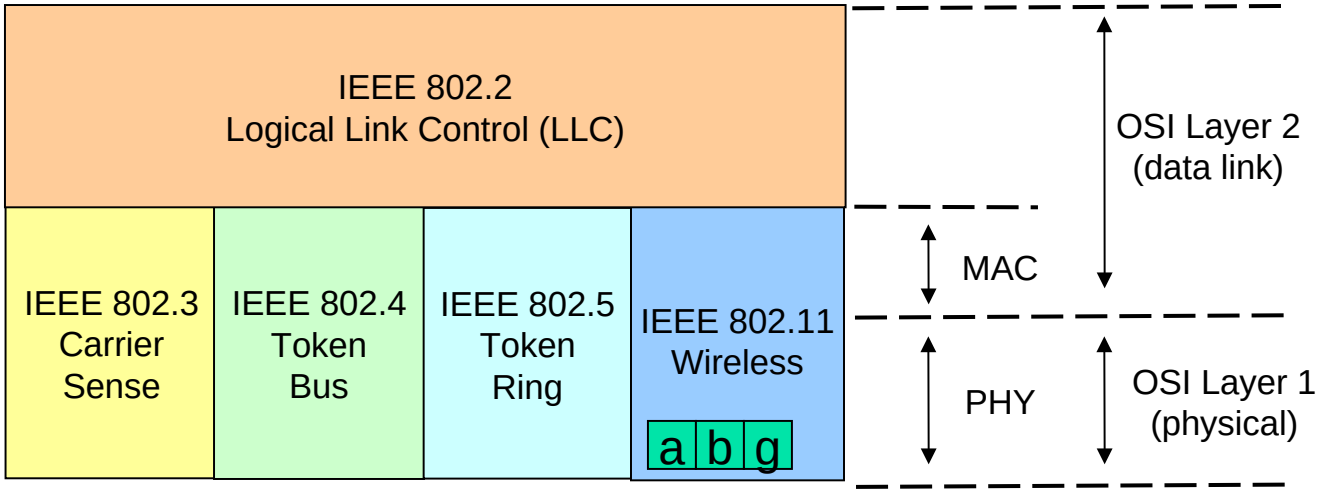
IEEE 802.12 ™: Demand Priority Access

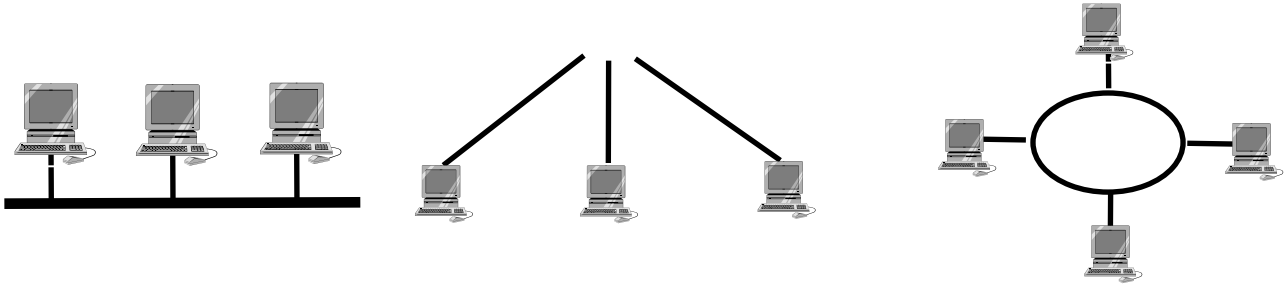
IEEE 802.16™: Broadband Wireless Metropolitan Area Networks

Demand priority: A round-robin (see token rings-later) arbitration method to provide LAN access based on message priority level

DQDB: Distributed queue dual buss, see PSTN lecture 2

The IEEE 802.11 and supporting LAN Standards



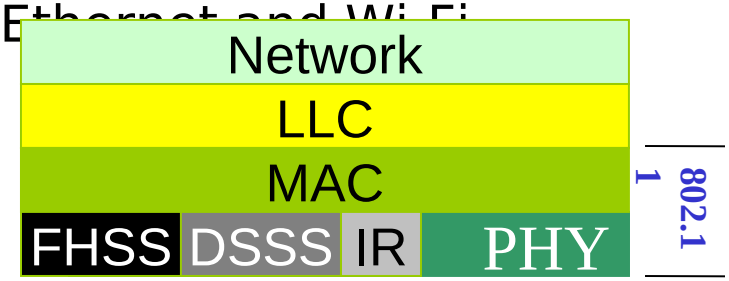


bus star ring

- See also IEEE LAN/MAN Standards Committee Web site www.manta.ieee.org/groups/802/
- IEEE 802.3 is Ethernet

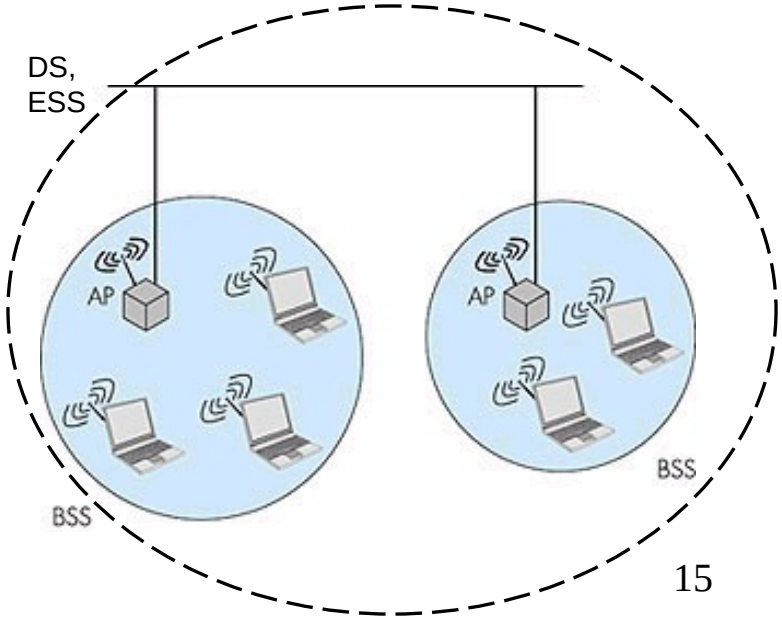
IEEE 802.11 Architecture

- IEEE 802.11 defines the physical (PHY), and media access control (MAC) layers for a wireless local area network; the logical link (LLC) is common to Ethernet and Wi-Fi
- 802.11 networks can work as
  - basic service set (BSS)
  - extended service set (ESS)
- BSS can also be used in ad-hoc networking

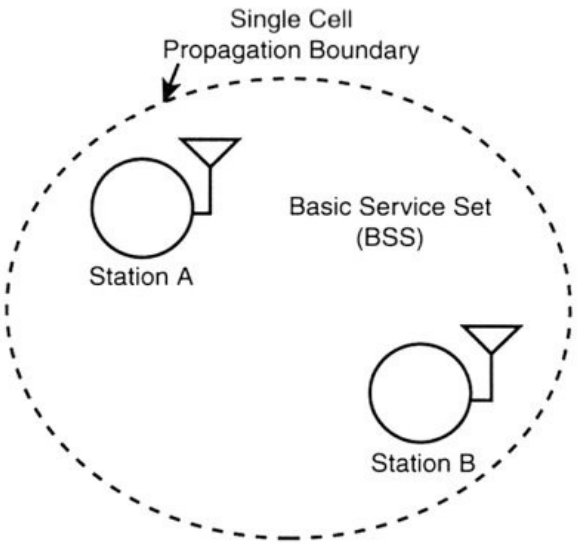


LLC: Logical Link Control Layer
MAC: Medium Access Control Layer
PHY: Physical Layer
FHSS: Frequency hopping SS
DSSS: Direct sequence SS
SS: Spread spectrum
IR: Infrared light
BSS: Basic Service Set
ESS: Extended Service Set
AP: Access Point
DS: Distribution System

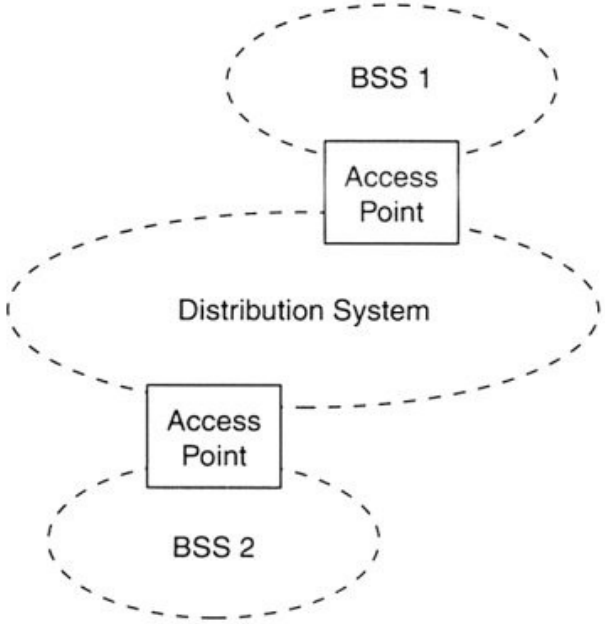
ad-hoc network



BSS and ESS



Basic (independent) service set (BSS)

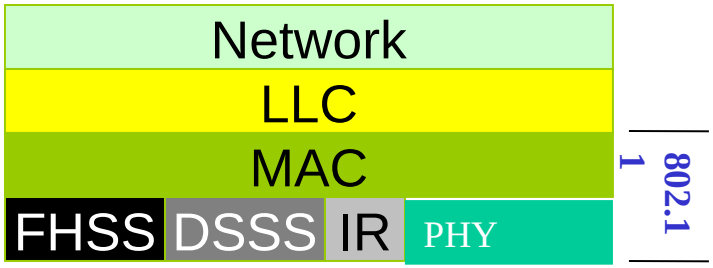


Extended service set (ESS)

- In ESS multiple access points connected by access points and a distribution system as Ethernet
  - BSSs partially overlap
  - Physically disjoint BSSs
  - Physically collocated BSSs (several antennas)

802.11 Logical architecture

- **LLC** provides
  - multiplexing mechanisms for several network protocols (IP, IPX, Decnet, e.g.)
  - flow control
  - automatic repeat requests (ARQ)
- **MAC** provides
  - Control access to wireless medium (CSMA/CA)
  - Addressing, frame delimiting/preamble for synchronization
  - protection against errors (frame check sequence)
- **Three physical layers (PHY)**
  - FHSS: Frequency Hopping Spread Spectrum (SS)
  - DSSS: Direct Sequence SS
  - IR: Infrared transmission

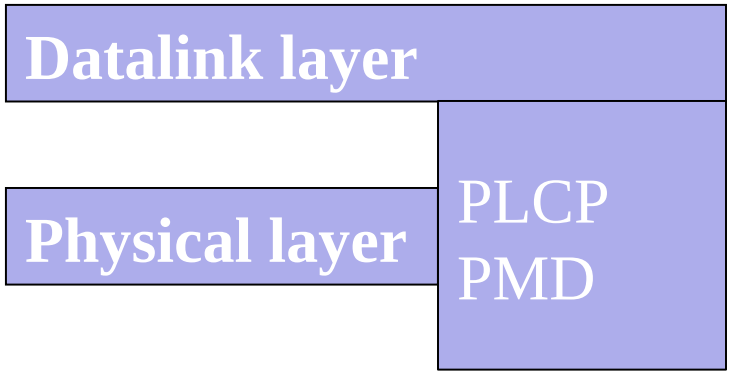


LLC: Logical Link Control Layer
MAC: Medium Access Control Layer
PHY: Physical Layer
FH: Frequency hopping
DS: Direct sequence
IR: Infrared light

802.11 Physical layer (PHY)

Physical layer structure

- Physical Layer Convergence Procedure (PLCP) sublayer: prepares MAC frames for transmission into the air; adds a preamble header to the MAC frame
- Physical Medium Dependent (PMD) sublayer : transmit (modulate) the bits into the air using an antenna

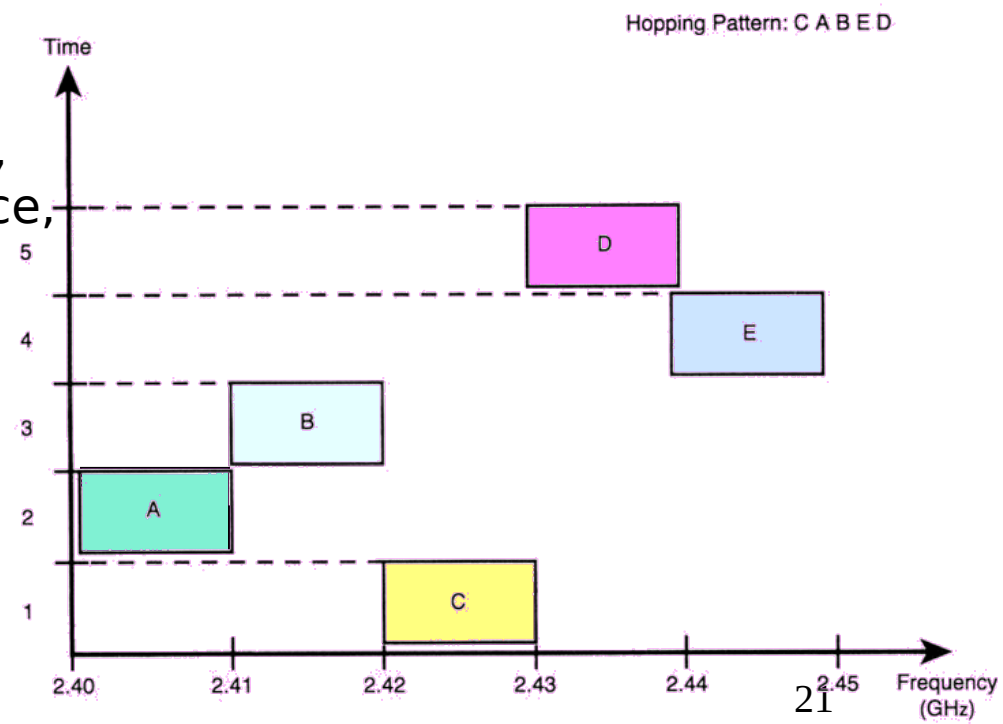


Spread spectrum techniques

- Spread spectrum works by using mathematical functions to diffuse signal power over a large set of frequencies
- It has the advantage that the emitted signal looks like noise and is less prone to interference
- Frequency hopping (FHSS) - jump from one frequency to another in a random pattern,transmitting a short burst at each subchannel. The 2-Mbps FH PHY is specified in clause 14
- Direct sequence (DSSS) - spread the power out over a wider frequency band using mathematical coding functions. Two direct-sequence layers were specified. The initial specification in clause 15 standardized a 2-Mbps PHY, and 802.11b added clause 18 for the HR/DSSS PHY.
- Orthogonal Frequency Division Multiplexing (OFDM) - divides an available channel into several subchannels and encodes a portion of the signal across each subchannel in parallel. The technique is similar to the Discrete Multi-Tone (DMT)technique used by some DSL modems. Clause 17, added with 802.11a, specifies the OFDM PHY. Clause 18, added in 802.11g, specifies the ERP PHY, which is essentially the same but operating at a lower frequency.

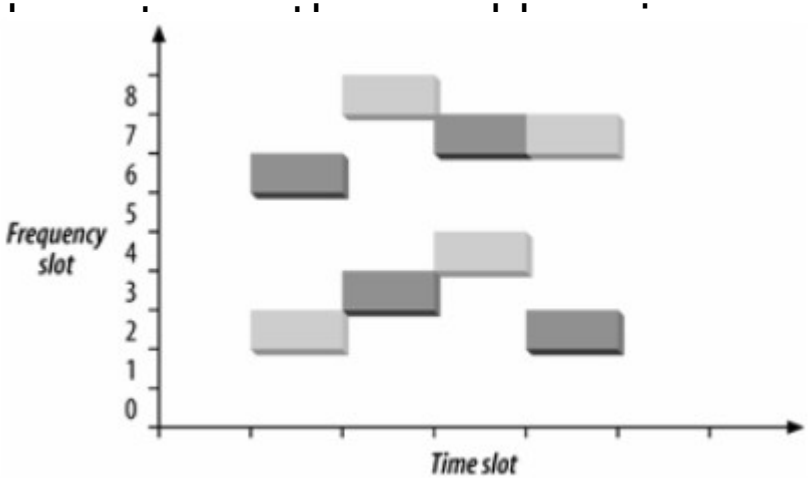
802.11 FHSS

- Supports 1 and 2 Mbps data transport and applies two level - GFSK modulation (Gaussian Frequency Shift Keying)
- 79 channels from 2.402 to 2.480 GHz (in U.S. and most of EU countries) with 1 MHz channel space
- 78 hopping sequences with minimum 6 MHz hopping space, each sequence uses every 79 frequency elements once
- Minimum hopping rate 2.5 hops/second
- Tolerance to multi-path, narrow band interference, security
- Low speed, small range due to FCC TX power regulation (10mW)



802.11 FHSS details

- 802.11 divides the ISM band into 1MHZ channels
- Modulation used in 802.11 shifts the transmission frequency from the channel center
- Channels are defined by their central frequency: channel 0 has 2.400GHz , channel 1 has 2.401GHz, channel 2 has 2.402GHz...
- The standard also defines sets of approx. 26 orthogonal hopping patterns (depending on the country)
- If the hopping patterns are orthogonal (i.e. they don’t overlap) for two users, there won’t be interferences between them
- In the following figure w
  patterns : {2, 8, 4, 7} a

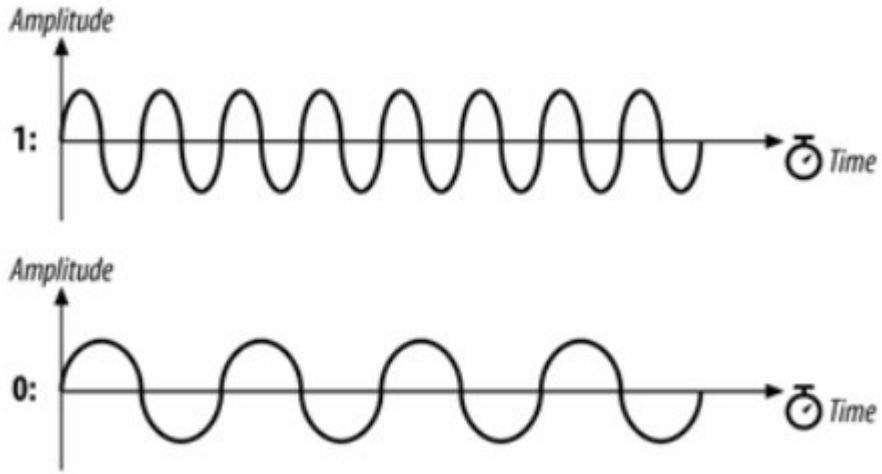


802.11 FHSS details (cont.)

- Beacon frames specify the hop pattern number (hop pattern are standardized), timestamp and current hop index
- A station can synchronize its frequency to the current hop specified by the beacon frame
- A station can spend 0.4 seconds in one hop channel
- Limitation: a channel has 1MHz bandwidth; 1 bit per cycle => 1Mbps maximum throughput

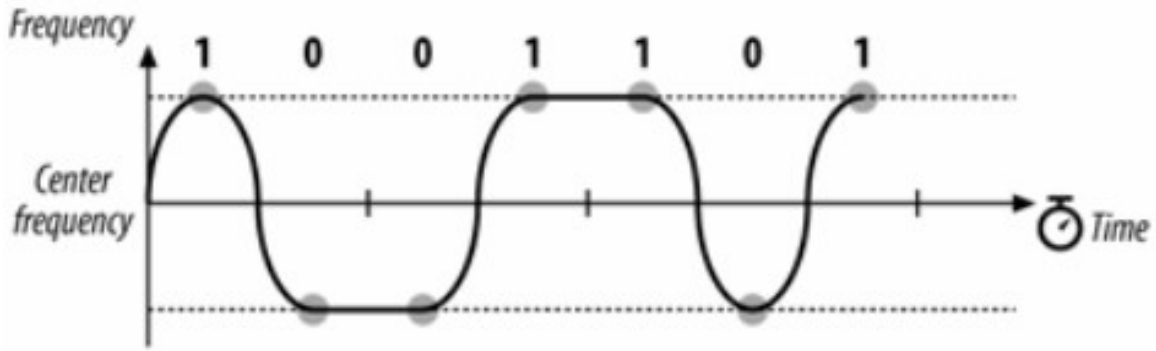
FHSS modulation. 2-level GFSK
(Gaussian Frequency Shift Keying)

- 2 frequencies are used for bit values 1 and 0: the central frequency increased by a level and the central frequency decreased by the same level



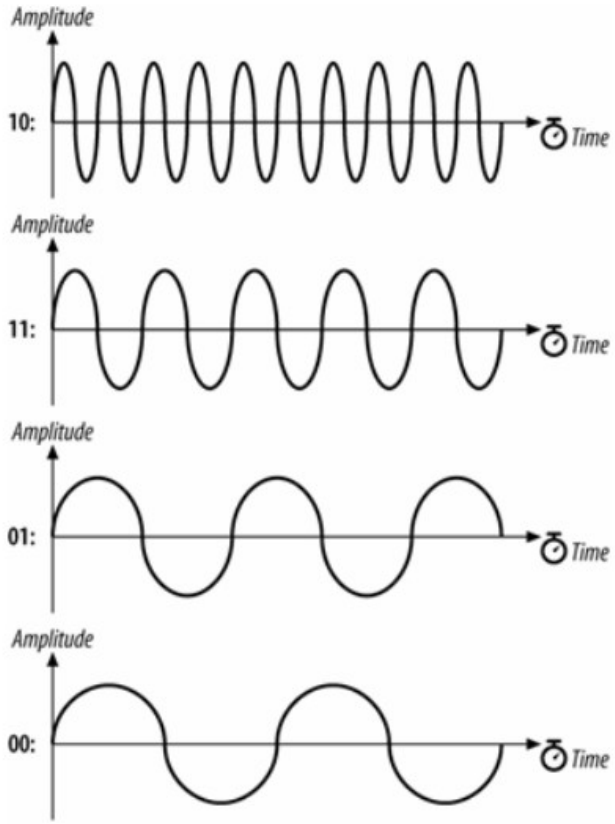
2-level GFSK modulation example

- Each symbol is modulated for a specific constant period of time
- In the middle of the symbol period, the receiver measures the frequency of the transmission and translates it into a symbol
- Usually the payload bit data is scrambled (whitened) before transmission
- Example of modulating the bit sequence 1001101:



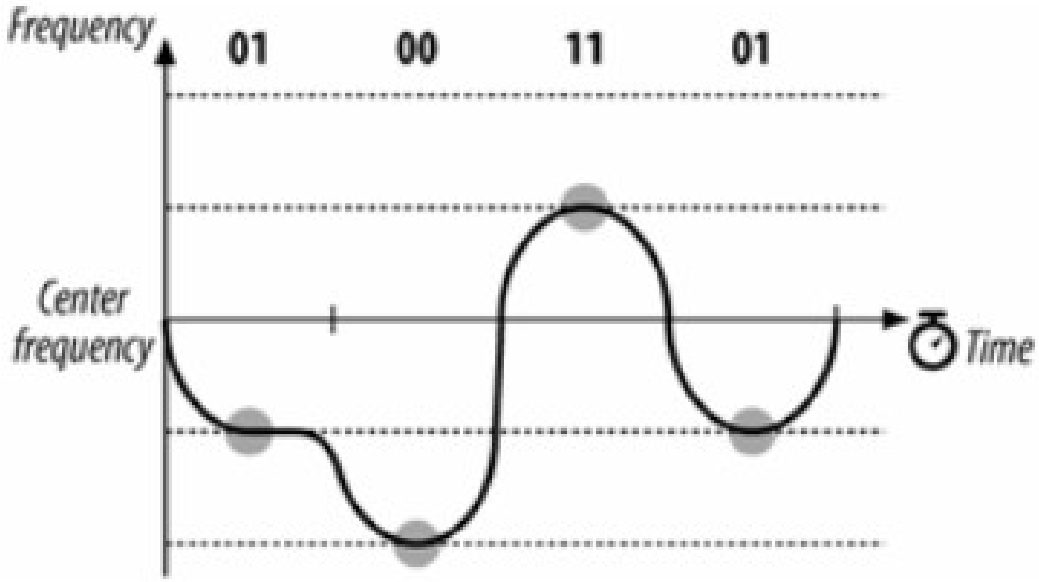
4-level GFSK

- There are 4 frequencies around the central frequency
- 4 symbols are encoded: 00, 01, 10, 11



4-level GFSK modulation example

- Encoding the same bit sequence 1001101



FHSS Physical Layer Convergence Procedure (PLCP) sublayer

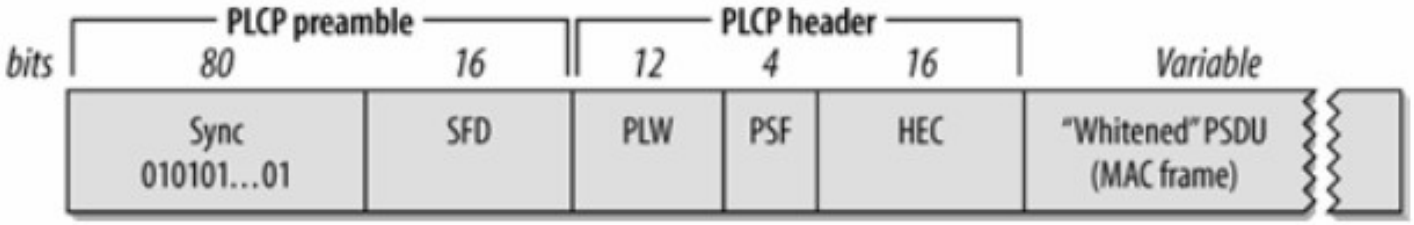
The PLCP sublayer adds a header+preamble to the MAC frame it receives:

•<u>Preamble</u> – it synchronizes the transmitter and receiver (like in Ethernet)

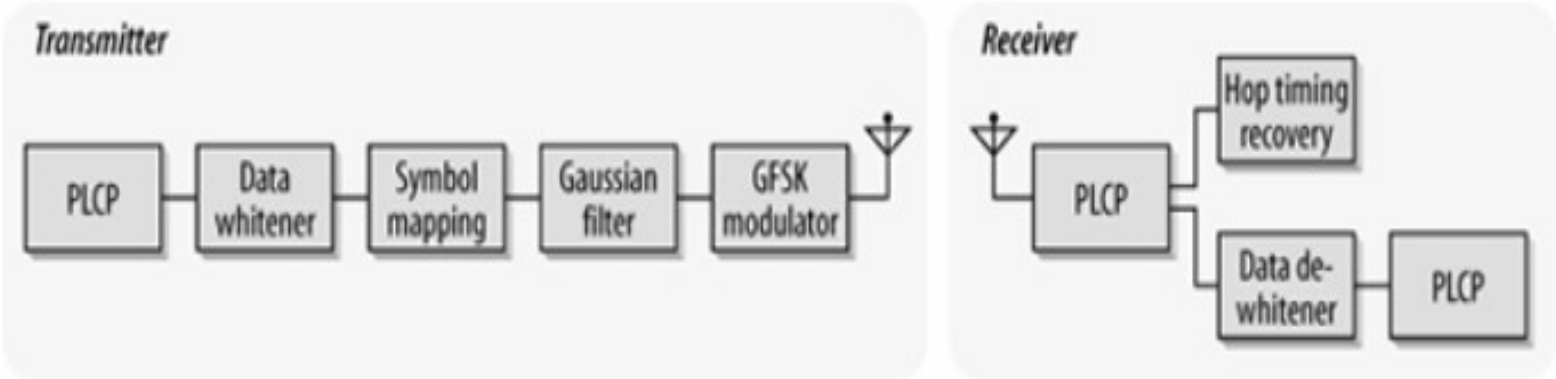
- Sync – 80 bits of alternating 0 and 1 (010101…); stations know that after the sync pattern they should prepare to receive data
- SFD (Start Frame Delimiter) – signals the end of preamble and beginning of a new frame; has the value: 0000 1100 1011 1101

•<u>Header</u>

- PSDU (PLCP Service Data Unit) Length Words (PLW) – the length of the MAC frame following this header (12 bits)
- PSF (PLCP Signaling) - bit 1 is zero, bits 2-4 encode the transmission speed of this frame(e.g. 001 = 1.5Mbps)

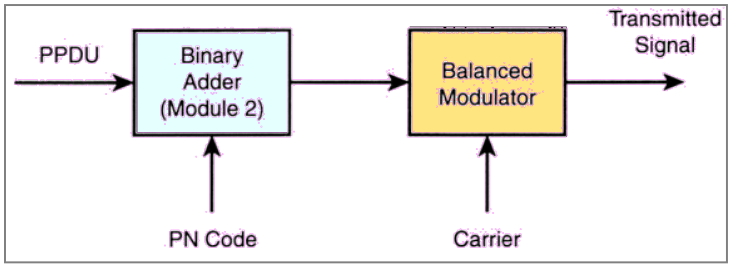


Frequency hopping transceiver (transmitter-receiver)

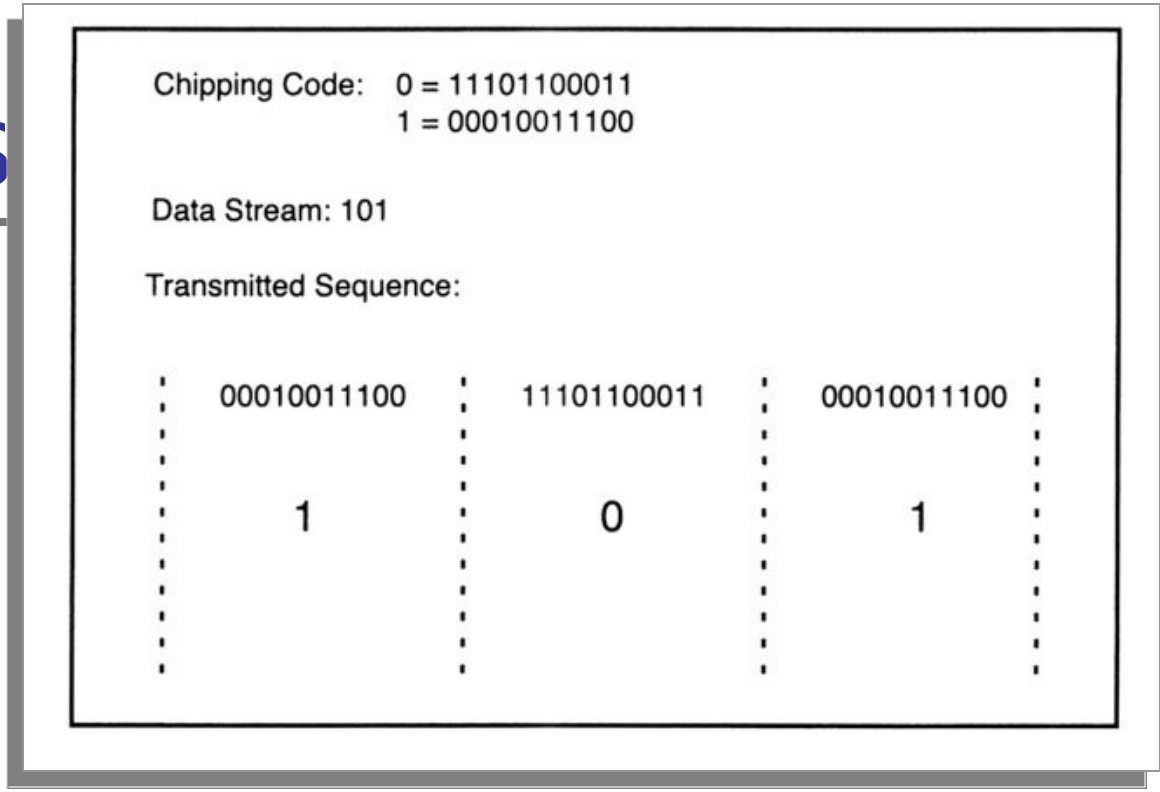


- the PLCP preamble is transmitted in 96us (preamble symbols are transmitted at 1MHz, so a symbol takes 1 us, to transmit; 96 bits require 96us)
- the PLCP header is transmitted in 32us (it has 32 bits, do it requires 32 symbol times)
- max size of a MAC frame 4095 bytes

802.11 DSS



DS-transmitter

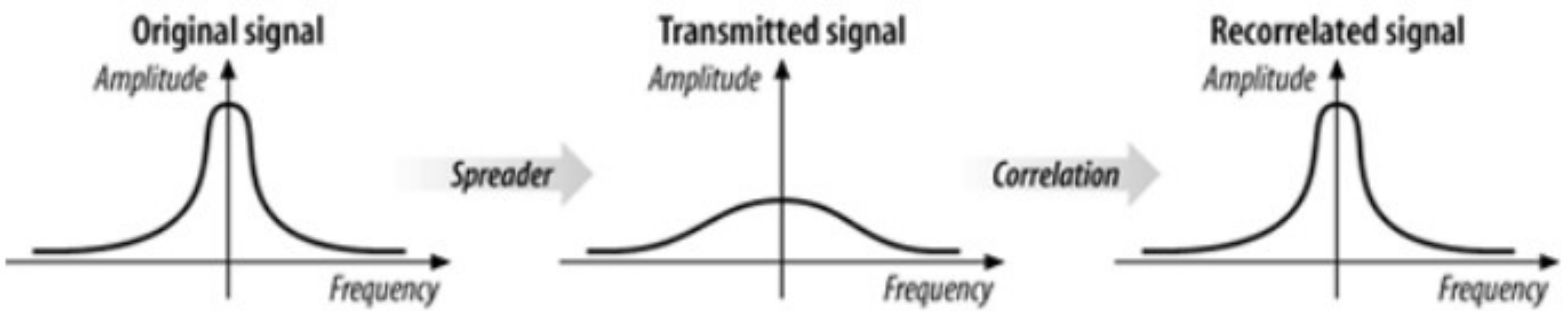


- Supports 1 and 2 Mbps data transport, uses BPSK and QPSK modulation (802.11b supports 5.5MHz and 11MHz)
- Uses 11 chips Barker (10110111000) code for spreading - 10.4 dB processing gain
- Defines 14 overlapping channels, each having 22 MHz channel bandwidth, from 2.401 to 2.483 GHz
- Power limits 1000mW in US, 100mW in EU, 200mW in Japan
- Immune to narrow-band interference, cheaper hardware

PPDU:baseband data frame

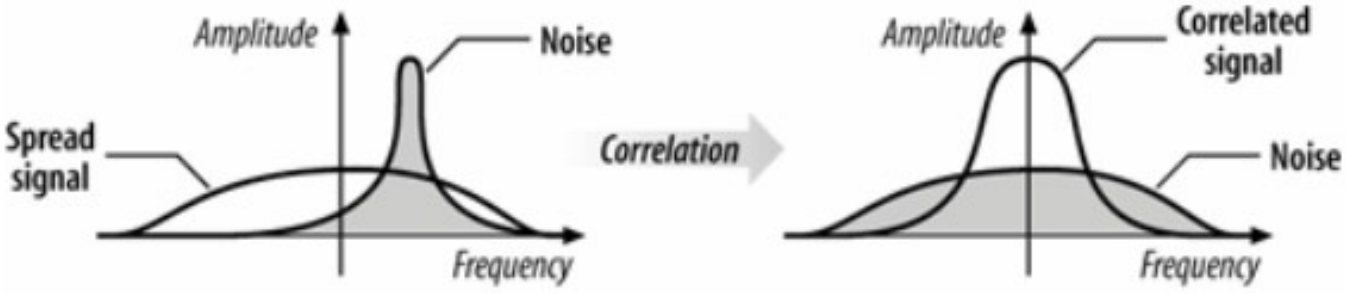
Direct Sequence Spread Spectrum technique

- The idea is to spread the signal over a wider frequency band
- This makes the transmitted signal look like low-level noise (i.e. more resilient to noise)
- The bitstream is modulated using a chiper that changes at a much higher rate than the original bitstream (the high speed oscillator required to produce the chip stream is a major power drain of



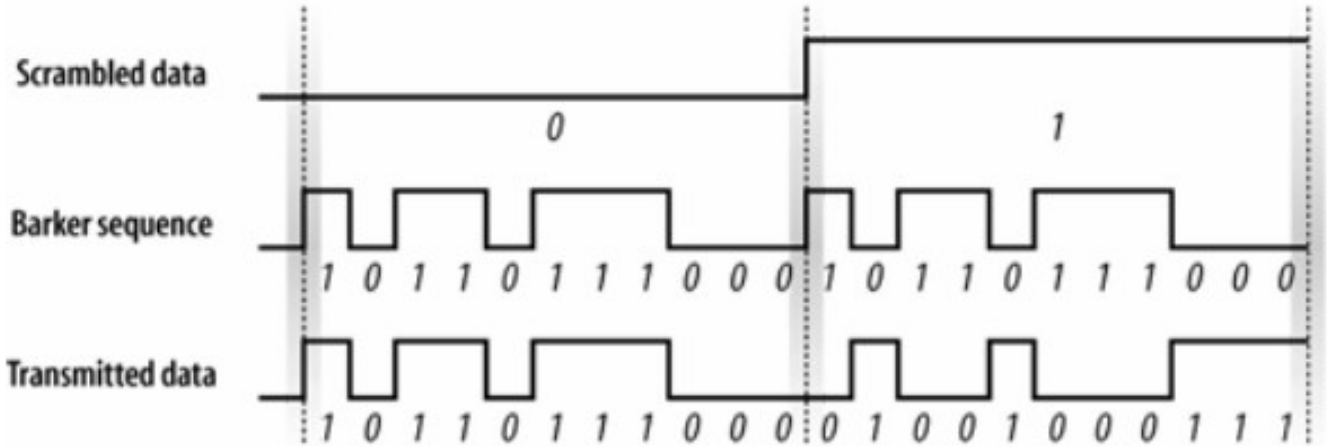
DSSS. Spreading the noise at receiver

- Noise tends to have the form of narrow pulses and does not produce effects across the entire frequency band
- The receiver spreads the noise and the signal is outlined



DSSS encoding example

- The Baker 11-bit chip word is: 10110111000
- This chip is modulo 2 added to the data bit

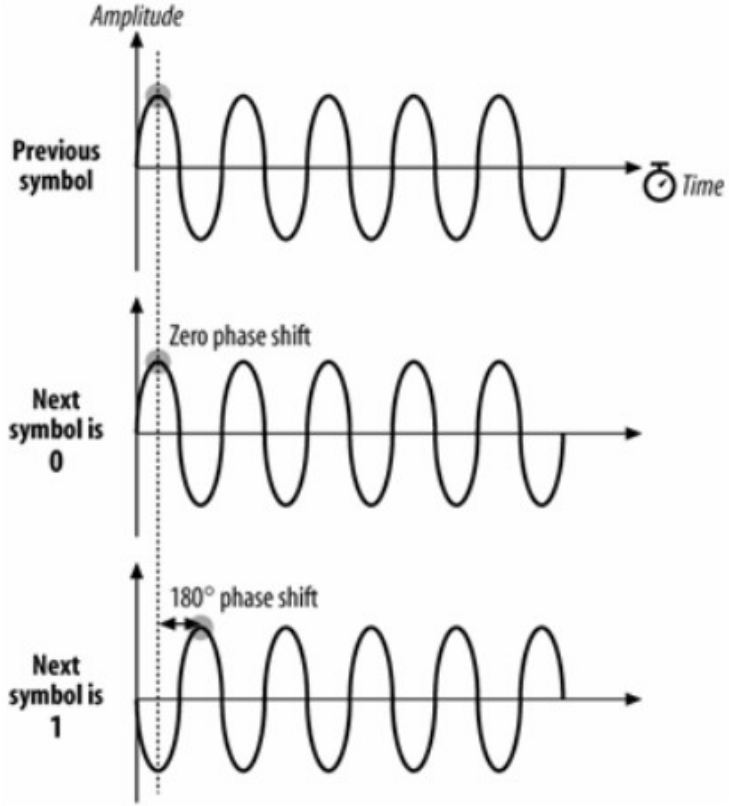


Radio spectrum in DSSS

- DSSS channels are much wider than FHSS channels
- DSSS physical level has 14 channels, each 5MHz wide
- Channel 1 has central freq. 2.412GHz, channel 2 has the central freq. 2.417GHz and .. channel 13 has the freq. 2.472GHz
- DSSS is more robust than FHSS
- Several bits need to be damaged in order for a single data bit to be lost.

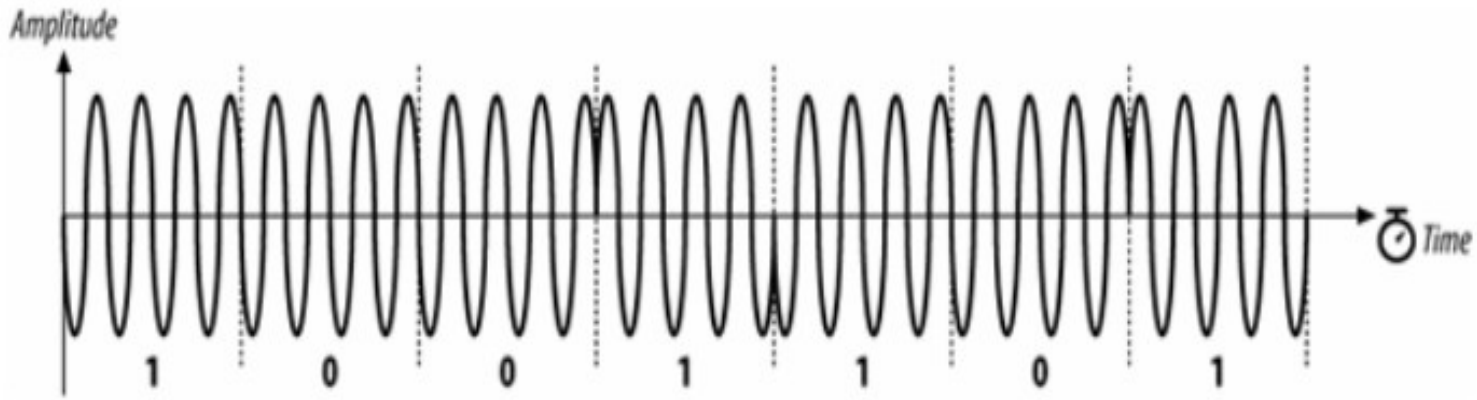
Modulation for DSSS. Differential Binary Phase Shift Keying (DBPSK)

- two carrier waves, one reference wave (phase 0) is used to encode a 0, a half-cycled shifted wave is used to encode a 1 (phase π)



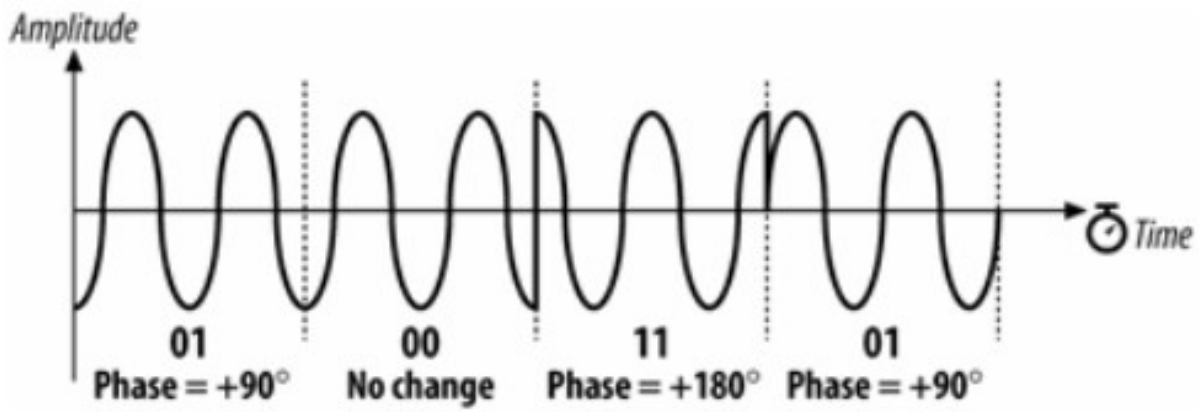
DBPSK example

- Encoding 1001101



Modulation for Boost Differential Quadrature Phase Shift Keying (DQPSK)

- 4 symbols:
  - 00 is encoded with a 0 phase shift,
  - 01 is encoded with a π/2 phase shift
  - 10 is encoded with a π phase shift
  - 11 is encoded with a 3 π/2 phase shift
- Encoding 01001101:



DSSS Physical Layer Convergence Procedure (PLCP) sublayer

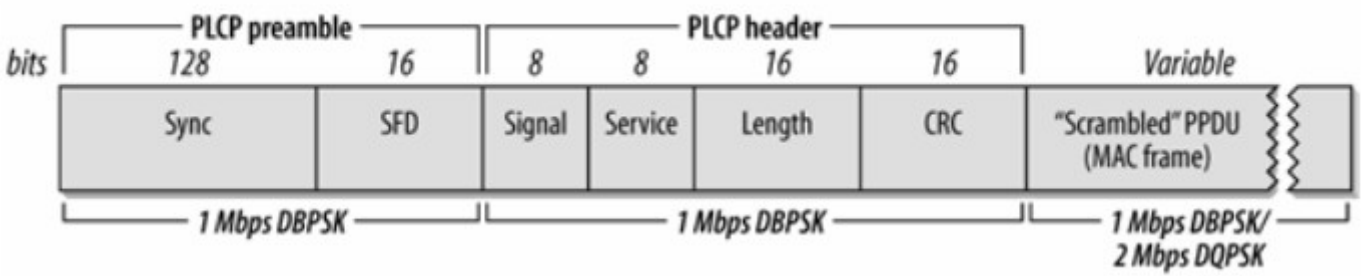
The PLCP sublayer adds a header+preamble to the MAC frame it receives:

•<u>Preamble</u> – it synchronizes the transmitter and receiver (like in Ethernet)

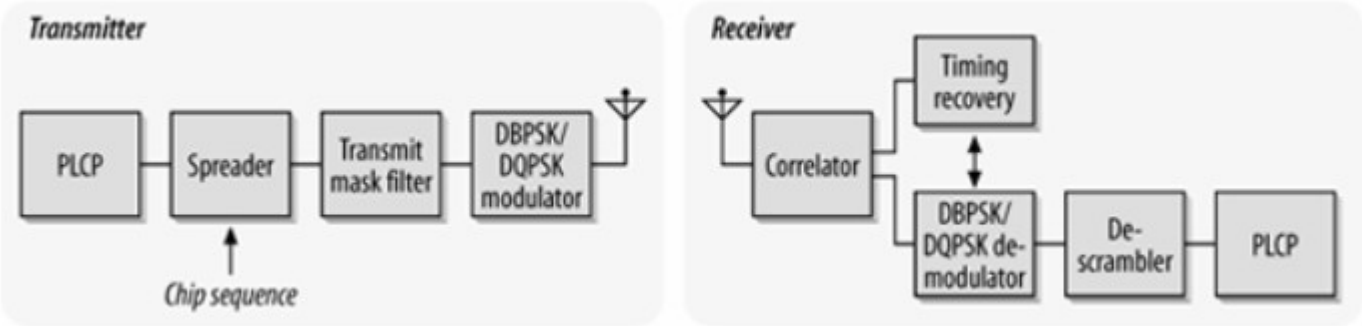
- Sync – 128 bits of value 1; stations know that after the sync pattern they should prepare to receive data
- SFD (Start Frame Delimiter) – signals the end of preamble and beginning of a new frame; has the value: 0000 0101 1100 1111; allows the receiver to find start of frame even if sync bits were lost

•<u>Header</u>

- Signaling - identified transmission speed bit: 0000 1010 = 1 Mbps operation, 0001 0100 = 2Mbps
- Service – all zeros
- Length – number of microseconds required to transmit the frame
- HEC (Header Error Check) – 16-bit CRC calculated over the header



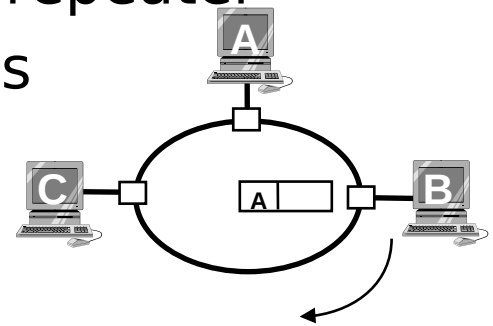
DSSS transceiver (transmitter-receiver)



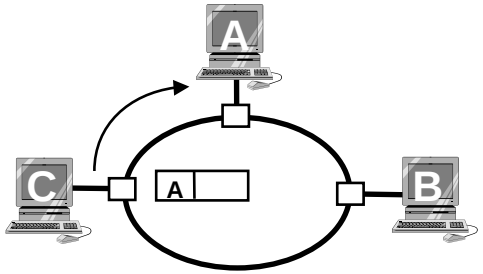
- the PLCP preamble is transmitted in 144us (preamble symbols are transmitted at 1MHz, so a symbol takes 1 us, to transmit; 144 bits require 144us)
- the PLCP header is transmitted in 48us (it has 48 bits, do it requires 48 symbol times)
- max size of a MAC frame 8191 bytes

How ring-network works

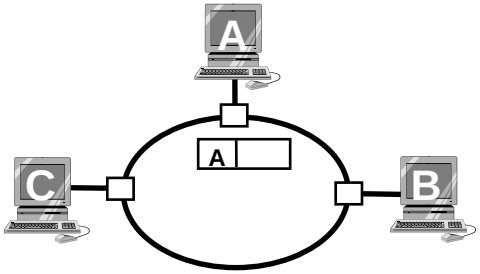
- A node functions as a repeater
- only destination copies frame to it, all other nodes have to discarded the frame
- Unidirectional link



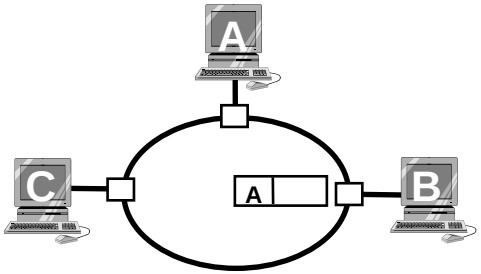
B transmits frame addressed to A



C ignores frame



A copies frame



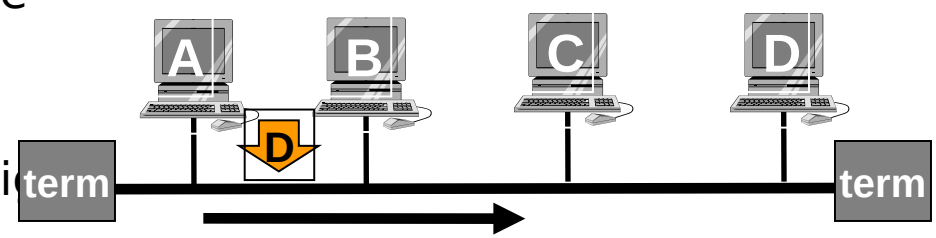
C absorbs returning frame

Token ring

- A ring consists of a single or dual (FDDI) cable in the shape of a loop
- Each station is only connected to each of its two nearest neighbors. Data in the form of packets pass around the ring from one station to another in uni-directional way.
- Advantages :
  - (1) Access method **supports heavy load** without degradation of performance because the medium is not shared.
  - (2) Several packets can simultaneous circulate between different pairs of stations.
- Disadvantages:
  - (1) Complex management
  - (2) Re-initialization of the ring whenever a failure occurs

How bus-network works

- In a bus network, one node's transmission traverses the entire network and is received and examined by every node. The access method can be :
  - (1) Contention scheme : multiple nodes attempt to access bus; only one node succeed at a time (e.g. CSMA/CD in Ethernet)
  - (2) Round robin scheme : a token is passed between nodes; node holds the token can use the bus (e.g.Token bus)
- Advantages:
  - (1) Simple access method
  - (2) Easy to add or remove stations
- Disadvantages:
  - (1) Poor efficiency with hi network load
  - (2) Relatively insecure, due to the shared medium

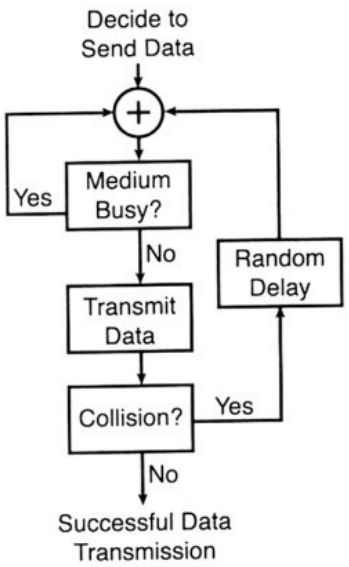


term: terminator impedance

MAC Techniques - overview

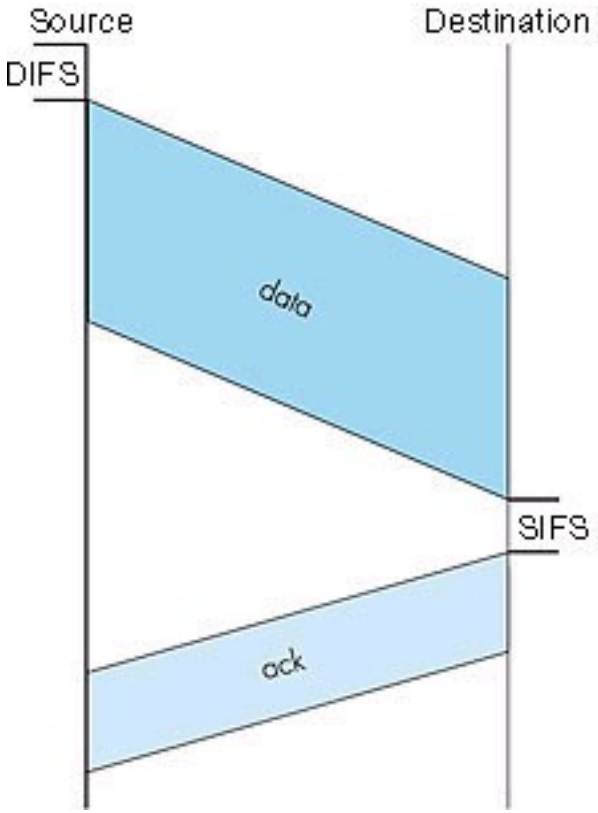
- Contention
  - Medium is free for all
  - A node senses the free medium and occupies it as long as data packet requires it
  - Example: Ethernet (CSMA), IEEE 802.3
- Token ring
  - Gives everybody a turn
  - reservation time depends on token holding time (set by network operator)
  - for heavy loaded networks
  - Example: Token Ring/IEEE 802.5, Token Bus/IEEE 802.4, FDDI
- Reservation (long term)
  - link reservation for multiple packets
  - Example: schedule a time slot: GSM using TDMA

IEEE 802.11 Media Access Control (MAC)



Carrier-sense multiple access protocol with collision avoidance (CSMA/CA)

1. An adapter may begin to transmit at any time, that is, no slots are used.
2. An adapter never transmits a frame when it senses that some other adapter is transmitting, that is, it uses carrier-sensing.
3. A transmitting adapter aborts its transmission as soon as it detects that another adapter is also transmitting, that is, it uses collision detection.
4. Before attempting a retransmission, an adapter waits a random time that is typically small compared to a frame time.



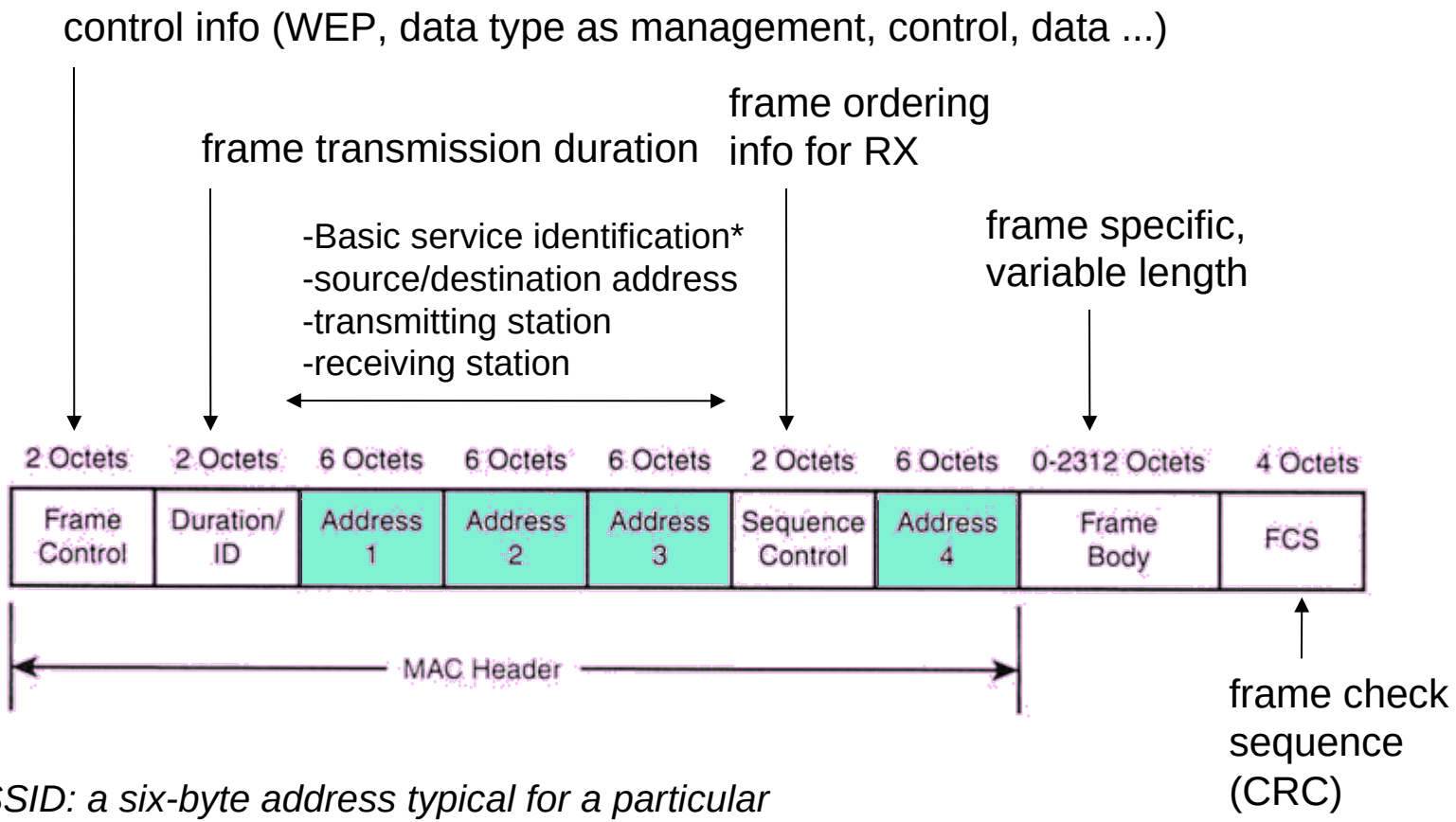
DIFS: Distributed Inter-Frame Spacing
SIFS: Short Inter-Frame Spacing
ack: Acknowledgement

IEEE 802.11 Media Access Control (cont.)

- There are various time intervals involved in 802.11 MAC CSMA/CA:
  - DIFS (DCF [Distributed Coordination Function] Interframe Space) – minimum idle time before a station can access the medium
  - SIFS (Short Interframe Space) – minimum idle time before a station can access the medium; used for highest-priority transmissions like RTS (Request to Send) and CTS (Clear to Send); SIFS < DIFS
- 802.11 MAC uses positive acknowledgement due to increased error rate of the wireless medium (each frame should be acknowledged by the receiver, otherwise retransmitted)
- A sender may send a RTS frame to inform other stations that it wants to transmit (no other station is allowed to transmit in this period) and all stations should reply with CTS – only in high contention environments

MAC frame (802.11 Wireless)

- NOTE: This frame structure is common for all data send by a 802.11 station



**BSSID: a six-byte address typical for a particular access point (network administrator sets)*

MAC Frame. Frame Control field

- Protocol version – 2 bits, value 00
- Type and Subtype – the type+subtype of frame (data frame or management):
  - Type =00 (management frame), subtypes: 0000 (association req.), 0001 (association response), 0100 (probe req.), 0101 (probe resp.), 1000 (beacon), 1010 (disassociation), 1011 (authentication)..
  - Type=01 (control frame), subtypes: 1010 (power save), 1011 (RTS), 1100 (CTS), 1101 (acknowledgement), ...
  - Type=10 (data frame), subtypes: 0000 (data), 0001 (Data+CF-Ack), ...
- To DS, From DS (DS=Distribution System, the wired network) – indicates whether the frame is from wireless to wired network or inverse

Protocol		Type=data		Sub type				To DS	From DS	More Frag	Retry	Pwr Mgmt	More Data	Protected Frame	Order
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

MAC Frame. Frame Control field (cont.)

- More Fragments bit – indicates that more fragments of this frame will follow
- Retry bit – this is a retransmitted frame
- Power Management bit – indicates if the sender will be in the powersaving mode after this send
- More data bit – indicates that more frames are buffered and ready to be transmitted (for powersaving receivers)
- Protected Frame bit – if this frame is encrypted (WEP, WPA, WPA2)

MAC Frame. Types of MAC addresses

- Destination address – the final recipient (station) of this frame
- Source address – the station that is the source of this frame
- Receiver address – either the final destination or the access point
- Transmitter address – the access point that transmitted this frame in the wireless network

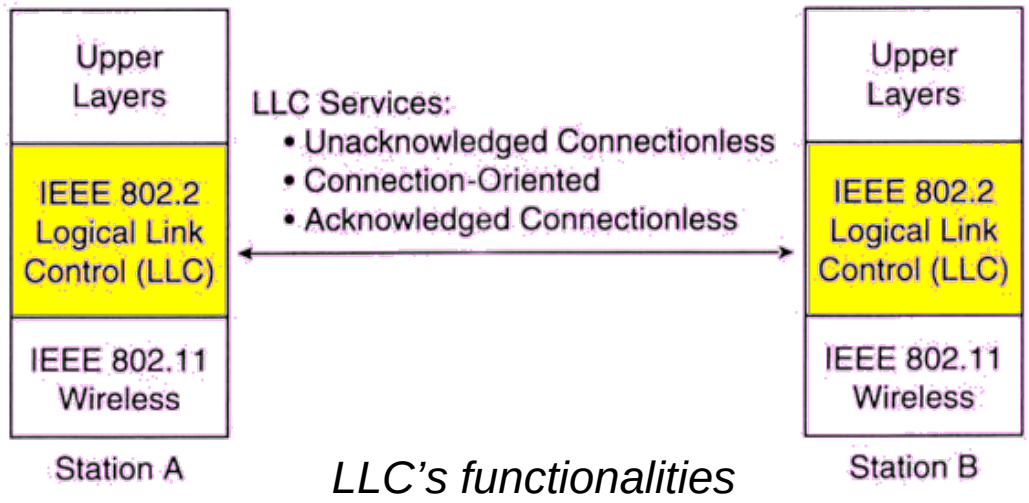
Mac Frame (802.3 Ethernet)

802.3 Ethernet frame structure

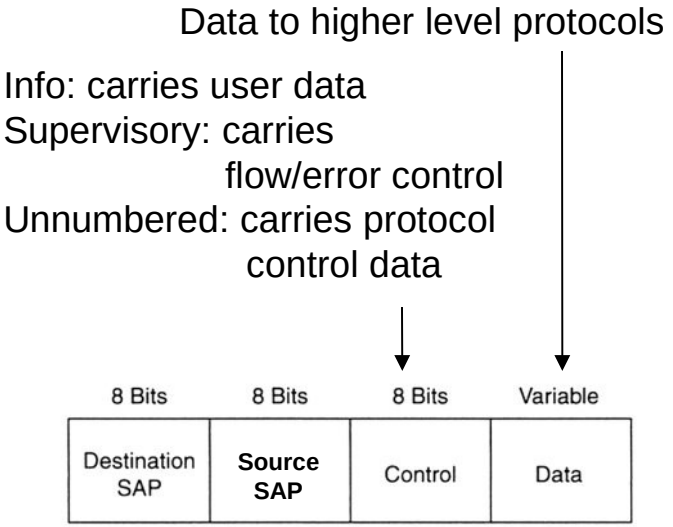
Preamble	Start of frame delimiter	MAC destination	MAC source	802.1Q tag (optional)	Ethertype or length	Payload	Cyclic redundancy check	Interframe gap
7 octets of 10101010	1 octet of 10101011	6 octets	6 octets	(4 octets)	2 octets	46–1500 octets	4 octets	12 octets
		64–1522 octets						
72–1530 octets								
84–1542 octets								

Logical Link Control Layer (LLC)

- Specified by ISO/IEC 8802-2 (ANSI/IEEE 802.2)
- purpose: exchange data between users across LAN using 802-based MAC controlled link
- provides addressing and data link control, independent of topology, medium, and chosen MAC access method



LLC's functionalities



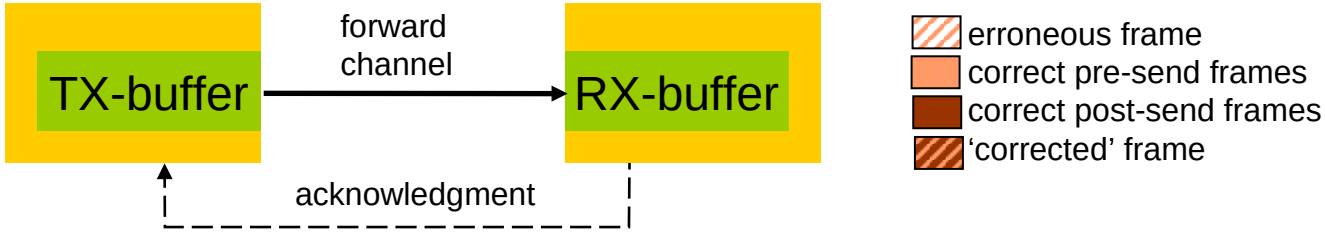
LLC's protocol data unit (PDU)
SAP: service address point

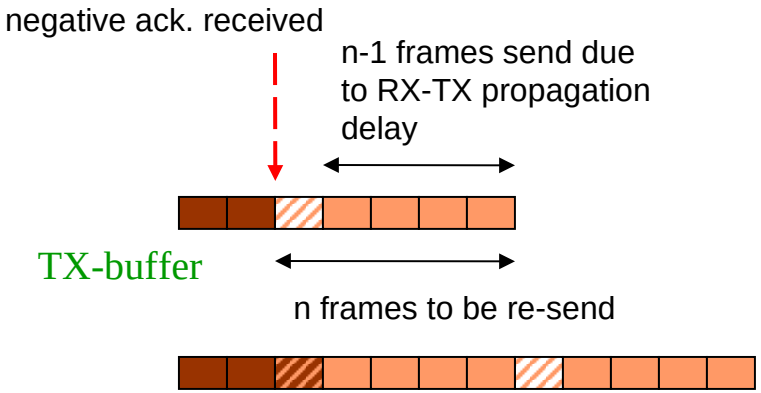
Logical Link Control Layer Services

- A **Unacknowledged connectionless** service
  - no error or flow control - no ack-signal usage
  - unicast (individual), multicast, broadcast addressing
  - higher levels take care or reliability - thus fast for instance for TCP
- B **Connection oriented** service
  - supports unicast only
  - error and flow control for lost/damaged data packets by cyclic redundancy check (CRC)
- C **Acknowledged connectionless** service
  - ack-signal used
  - error and flow control by stop-and-wait ARQ
  - faster setup than for B

ARQ (Automatic Repeat Request) Techniques (acknowledgement/retransmission)

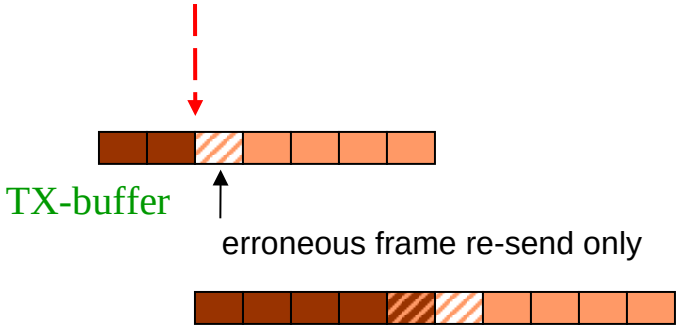
ARQ-system:





Go-back-n

- also correct frames re-send
- small receiver buffer size enough
- no reordering in RX



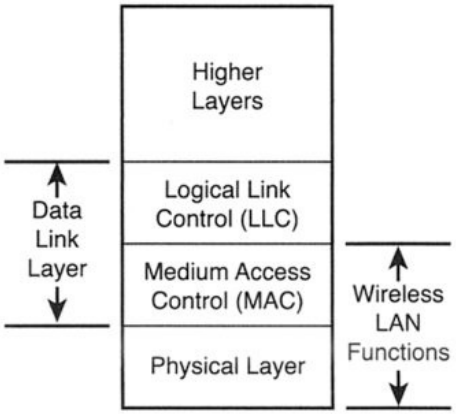
Selective repeat

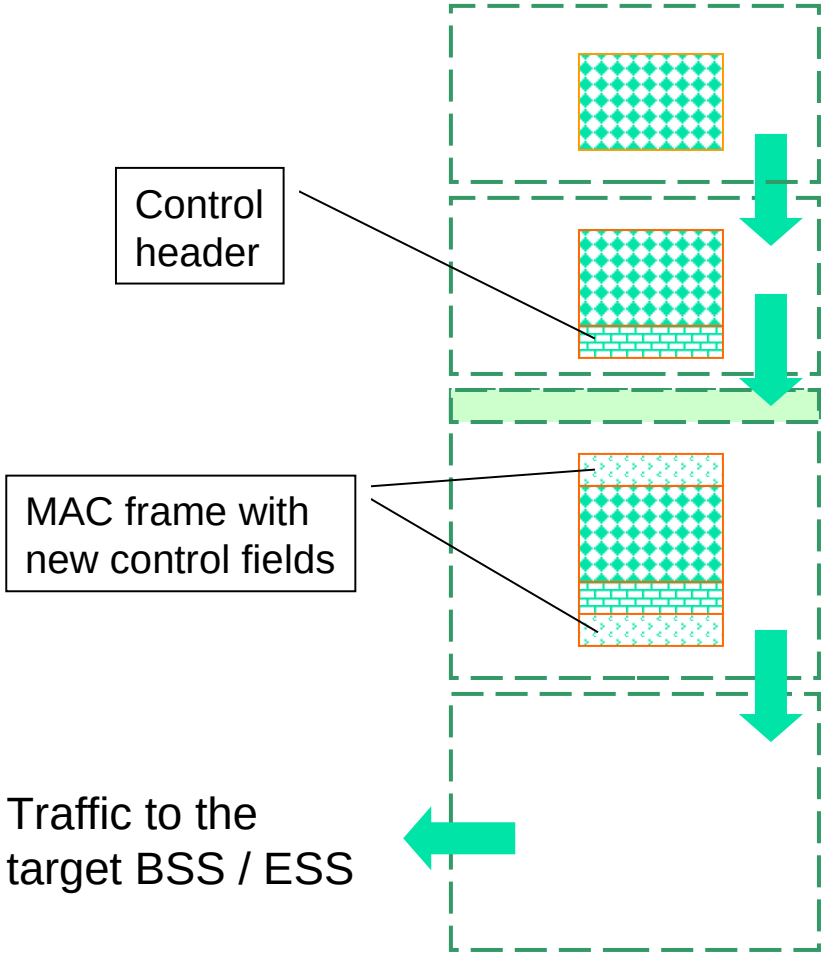
- reordering might be required in RX
- large buffer required in TX

Stop-and-wait

- for each packet wait for ack.
- if negative ack received, re-send packet
- inefficient if long propagation delays

A TCP/IP packet in 802.11





TPC/IP send data packet

LLC constructs PDU by adding a control header

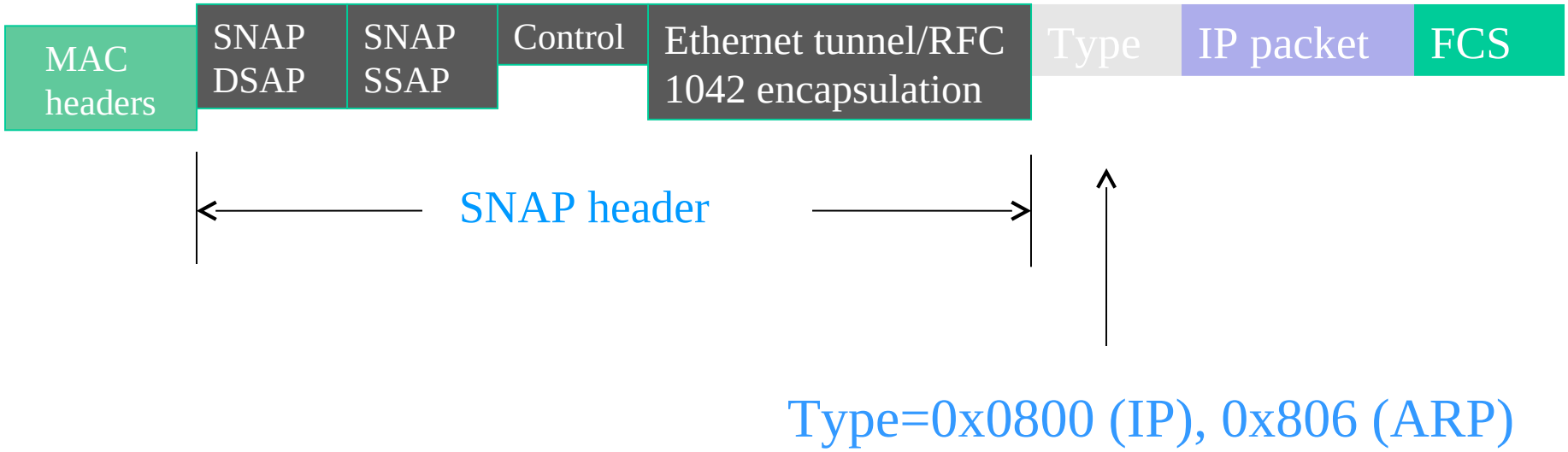
SAP (service access point)

MAC lines up packets using carrier sense multiple access (CSMA)

PHY layer transmits packet using a modulation method (DSSS, OFDM, IR, FHSS)

**PDU: protocol data unit*

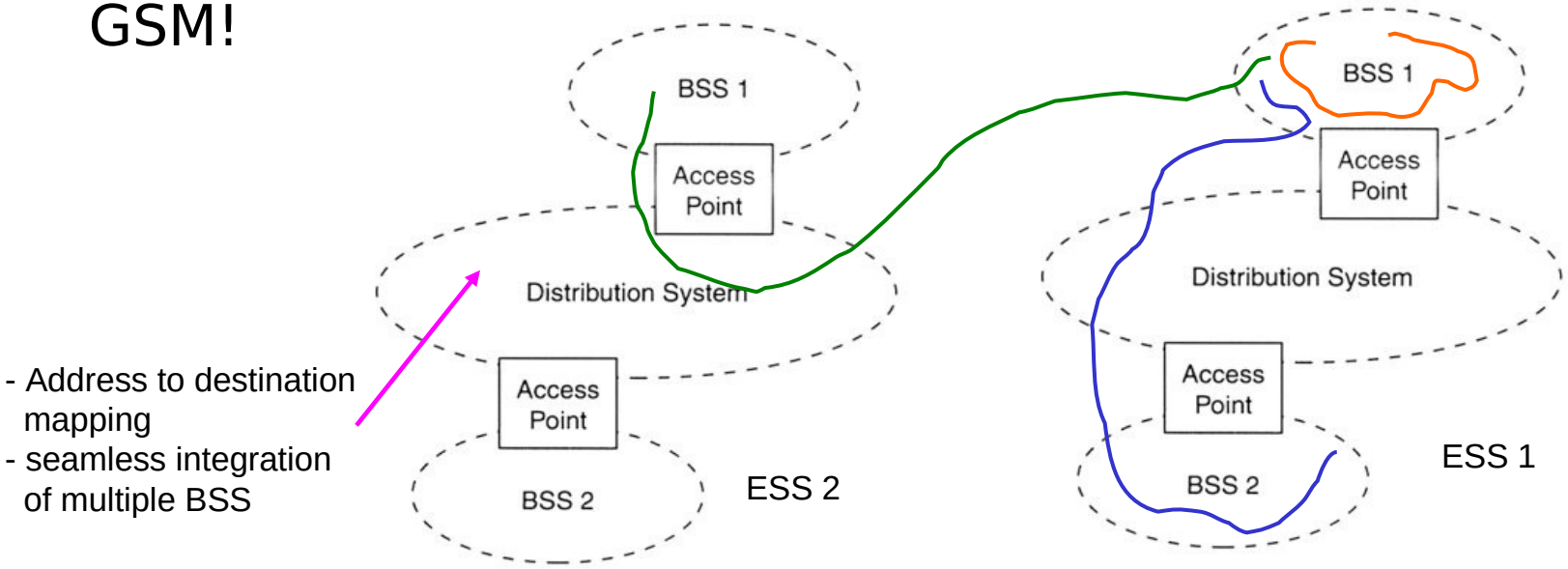
Encapsulation of IP packet in 802.11 LLC+MAC frame



Type=0x0800 (IP), 0x806 (ARP)

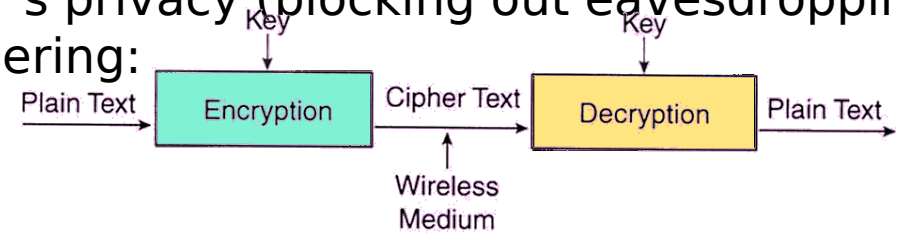
IEEE 802.11 Mobility

- Standard defines the following mobility types:
  - No-transition: no movement or moving within a local BSS
  - BSS-transition: station movies from one BSS in one ESS to another BSS within the same ESS
  - ESS-transition: station moves from a BSS in one ESS to a BSS in a different ESS (continuos roaming not supported)
- Especially: 802.11 don't support roaming with GSM!



Authentication and privacy

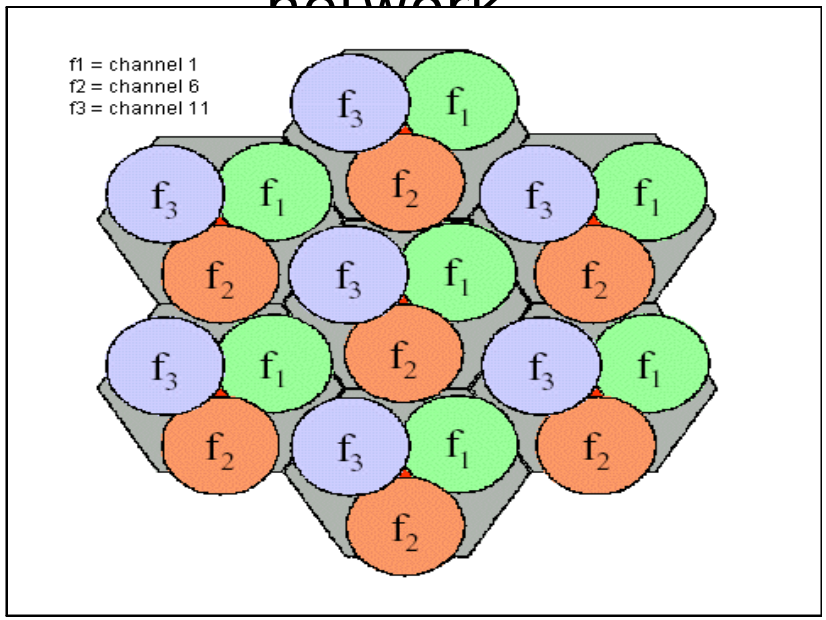
- Goal: to prevent unauthorized access & eavesdropping
- Realized by authentication service prior access
- <u>Open system authentication</u>
  - station wanting to authenticate sends **authentication management frame** - receiving station sends back frame for successful authentication
- <u>Shared key authentication</u> (included in WEP*)
  - **Secret, shared key** received by all stations by a separate, 802.11 independent channel
  - Stations authenticate by a shared knowledge of the key properties
- WEP's privacy (blocking out eavesdropping) is based on ciphering:

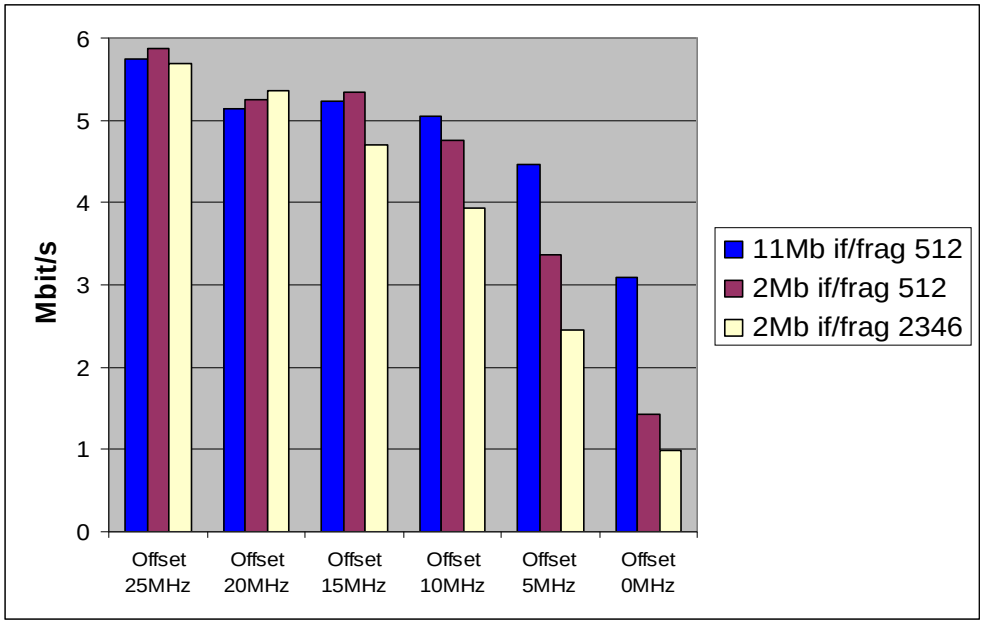


*WEP: Wired Equivalent Privacy

Frequency planning

- Interference from other WLAN systems or cells
- IEEE 802.11 operates at uncontrolled ISM band
- 14 channels of 802.11 are overlapping, only 3 channels are disjointed. For example Ch1, 6, 11
- Throughput decreases with less channel spacing
- A example of frequency allocation in multi-cell network





WLAN benefits

- Mobility
  - increases working efficiency and productivity
  - extends the On-line period
- Installation on difficult-to-wire areas
  - inside buildings
  - road crossings
- Increased reliability
  - Note: Pay attention to security!
- Reduced installation time
  - cabling time and convenient to users and difficult-to-wire cases

WLAN benefits (cont.)

- Broadband
  - 11 Mbps for 802.11b
  - 54 Mbps for 802.11a/g (GSM:9.6Kbps, HCSCD:~40Kbps, GPRS:~160Kbps, WCDMA:up to 2Mbps)
- Long-term cost savings
  - O & M cheaper that for wired nets
  - Comes from easy maintenance, cabling cost, working efficiency and accuracy
  - Network can be established in a new location just by moving the PCs!

WLAN technology problems

- Date Speed
  - IEEE 802.11b support up to 11 MBps, sometimes this is not enough - far lower than 100 Mbps fast Ethernet
- Interference
  - Works in ISM band, share same frequency with microwave oven, Bluetooth, and others
- Security
  - Current WEP algorithm is weak - usually not ON!
- Roaming
  - No industry standard is available and propriety solution are not interoperable - especially with GSM
- Inter-operability
  - Only few basic functionality are interoperable, other vendor’s features can’t be used in a mixed network

WLAN implementation problems

- Lack of wireless networking experience for most IT engineer
- No well-recognized operation process on network implementation
- Selecting access points with ‘Best Guess’ method
- Unaware of interference from/to other networks
- Weak security policy
- As a result, your WLAN may have
  - Poor performance (coverage, throughput, capacity, security)
  - Unstable service
  - Customer dissatisfaction

SDN – Software Defined Networking

Learning objectives

- Understand the difference between traditional networking and SDN
- Learn SDN architecture and key components
- Explore SDN controllers and protocols
- Simulating/experimenting with SDN
- Discuss challenges and future directions

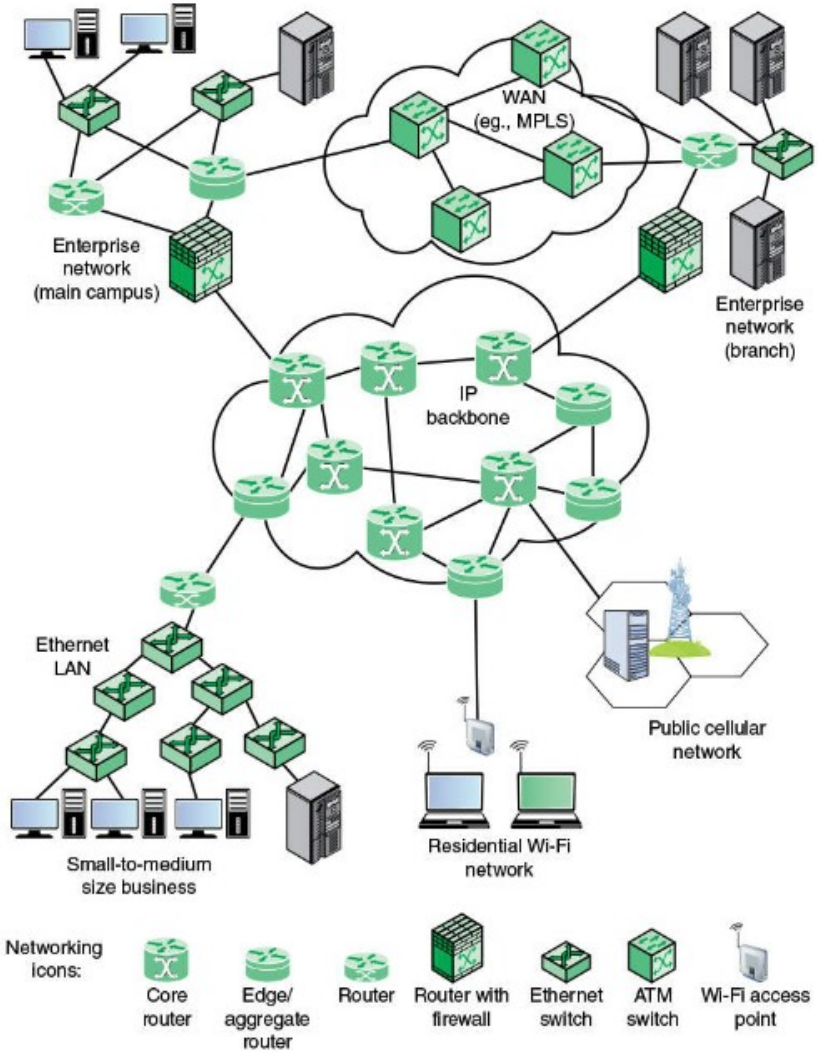
Why SDN ?

- Traditional networks are hardware-centric and vendor-specific
- Complex to configure and scale
- Slow to respond to changes or failures
- SDN enables **programmability, automation, and centralized control**

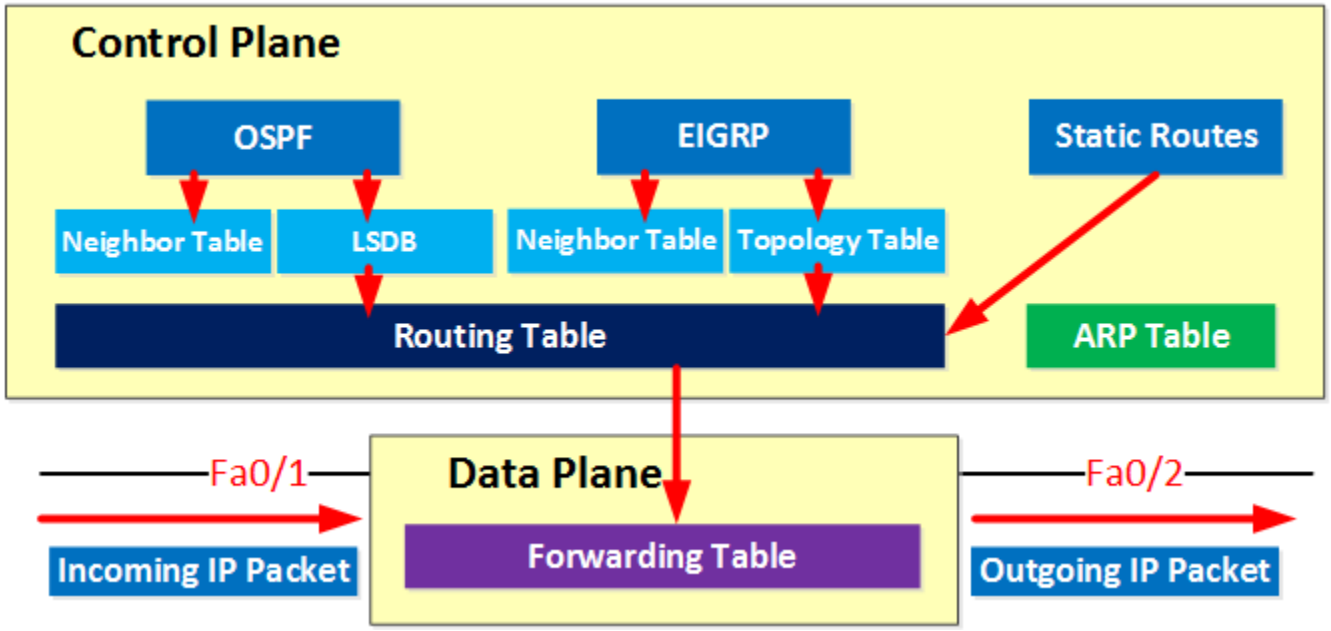
Traditional vs. SDN Networking

- Traditional:
  - distributed control plane, manual configuration
  - static, complex architecture
  - Inconsistent policies
  - inability to scale
  - vendor dependence
- SDN:
  - centralized control plane, software-defined policies
  - decouples **control plane** and **data plane**
  - adaptability and maintainability
  - automation
  - on-demand scaling

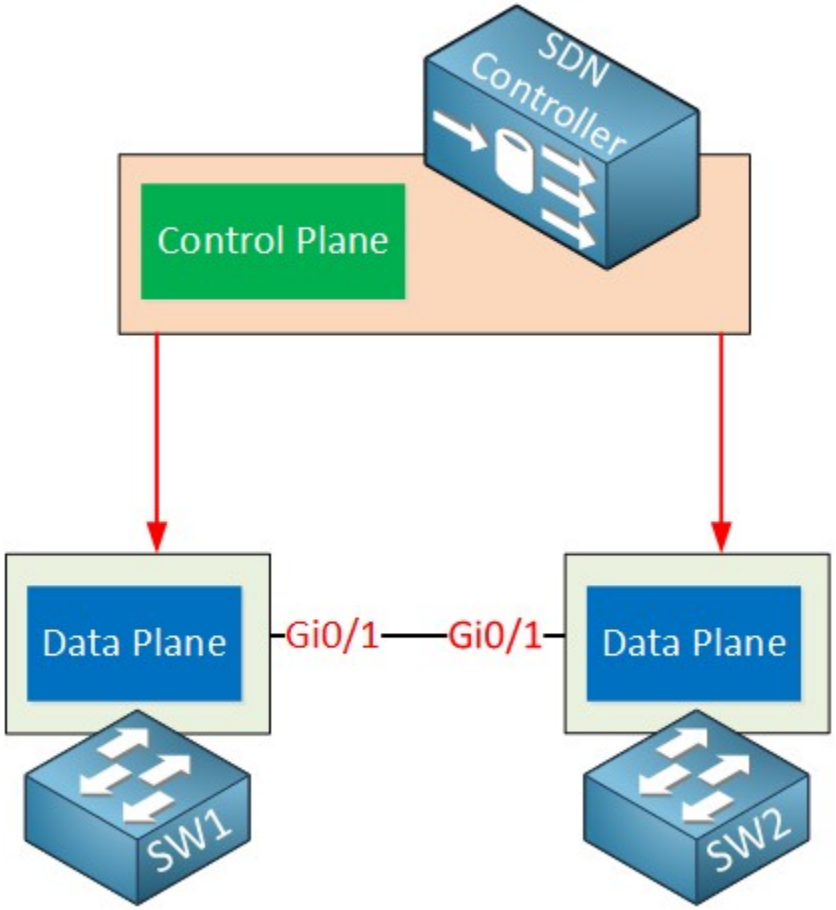
Typical network architecture (traditional and SDN)



Traditional networking



Software Defined Networking



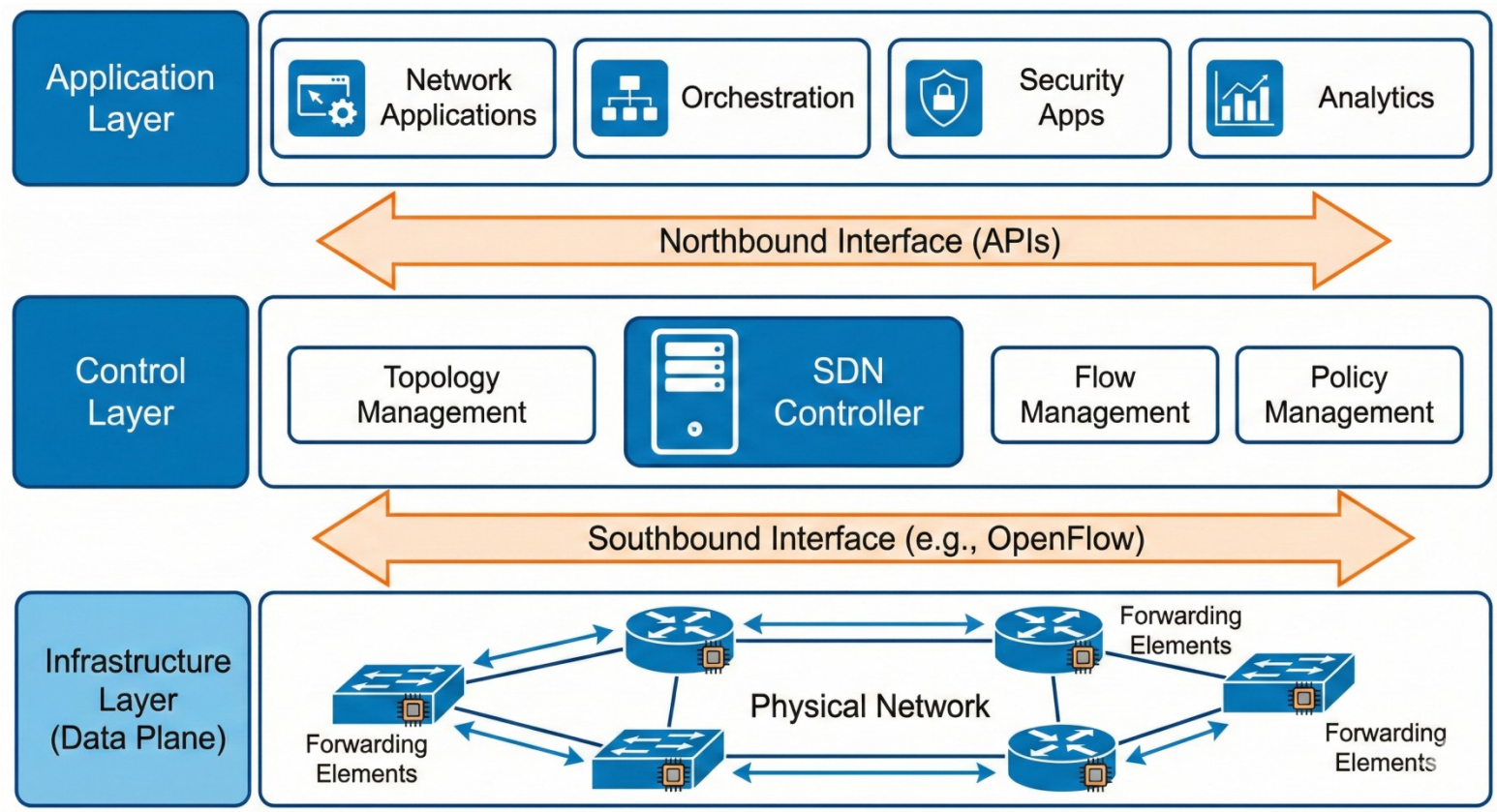
SDN Core Principles

- Separation of control and data plane
- Programmable network behavior
- Centralized network intelligence
- Open interfaces (APIs) for automation
- SDN makes networks “software-controllable” like cloud systems

SDN Architecture Overview

- Infrastructure Layer – switches, routers
- Control Layer – SDN controller, network OS
- Application Layer – apps using network APIs

SDN architecture



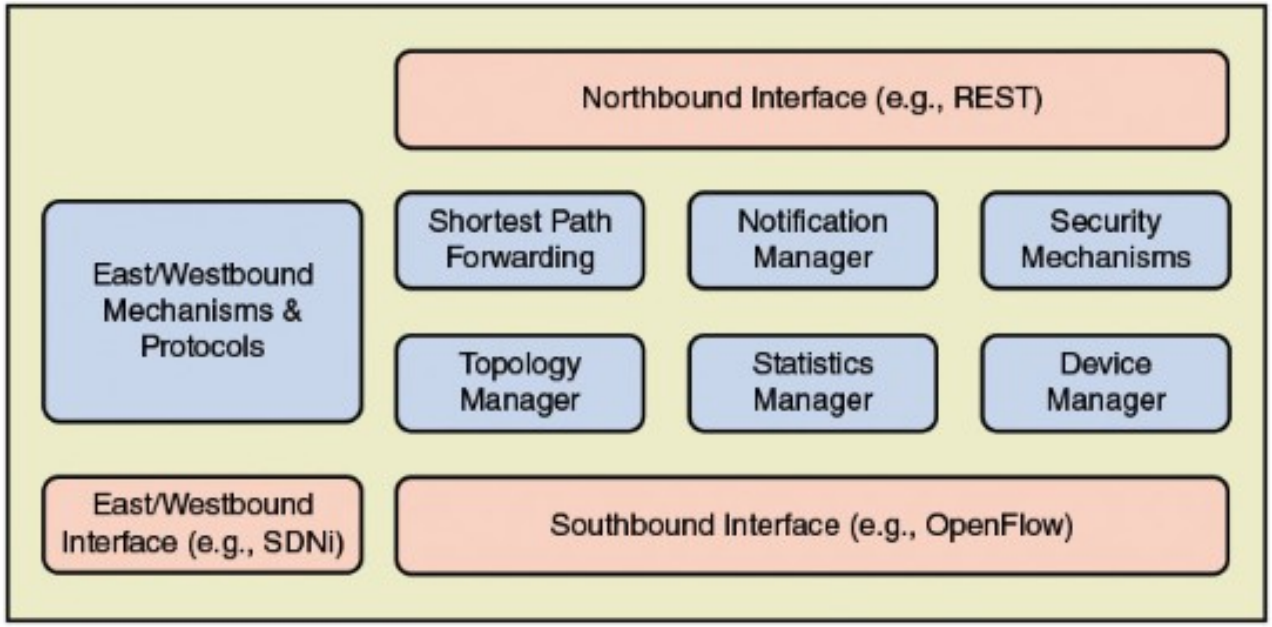
SDN Infrastructure Layer

- Physical or virtual switches
- OpenFlow-capable devices
- Forwarding devices execute controller instructions

SDN Control Layer

- Centralized SDN Controller
- Maintains global network view
- Computes paths and distributes flow rules
- Examples: ONOS, OpenDaylight, Ryu, Floodlight

SDN Controller functions



SDN Application Layer

- Network applications use **northbound APIs**
- Examples:
  - Traffic engineering
  - Security automation
  - QoS and monitoring
- apps can dynamically react to network state

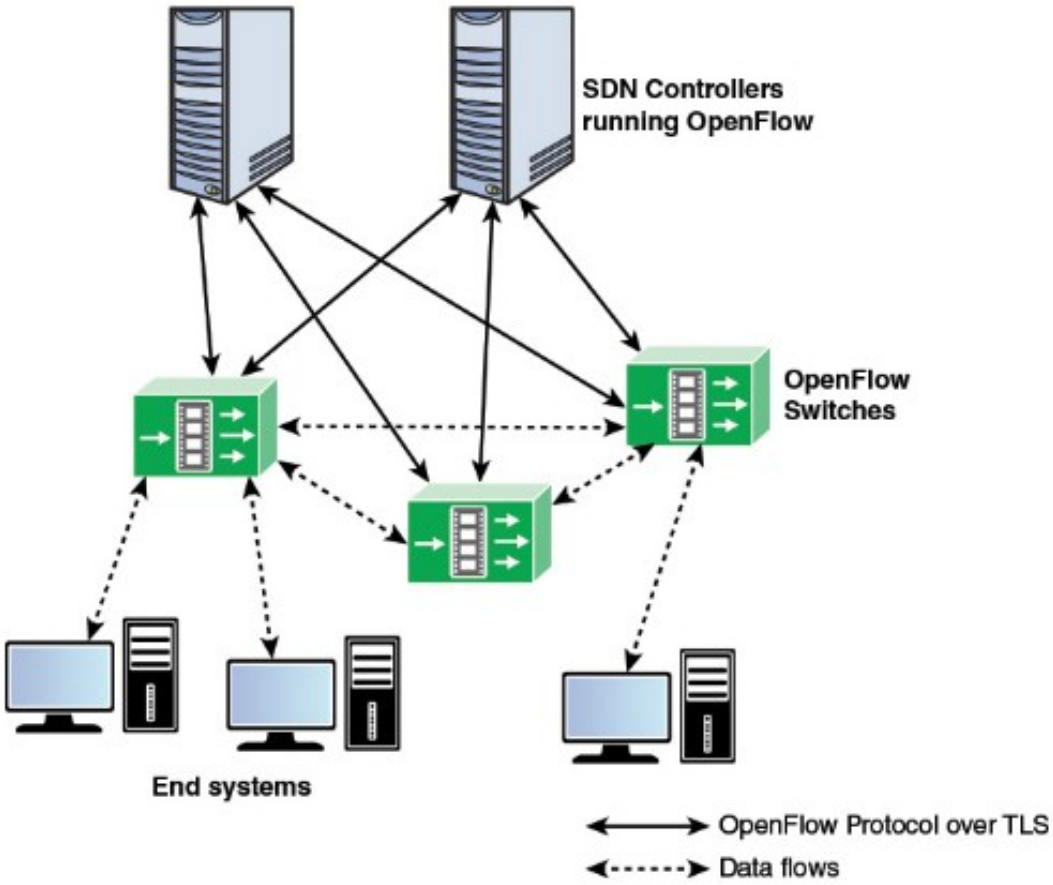
SDN APIs

- **Southbound APIs:** OpenFlow, NETCONF/YANG, P4Runtime
- **Northbound APIs:** REST, intent-based interfaces, higher-level orchestration
- Southbound APIs control switches; northbound API enable apps to control network

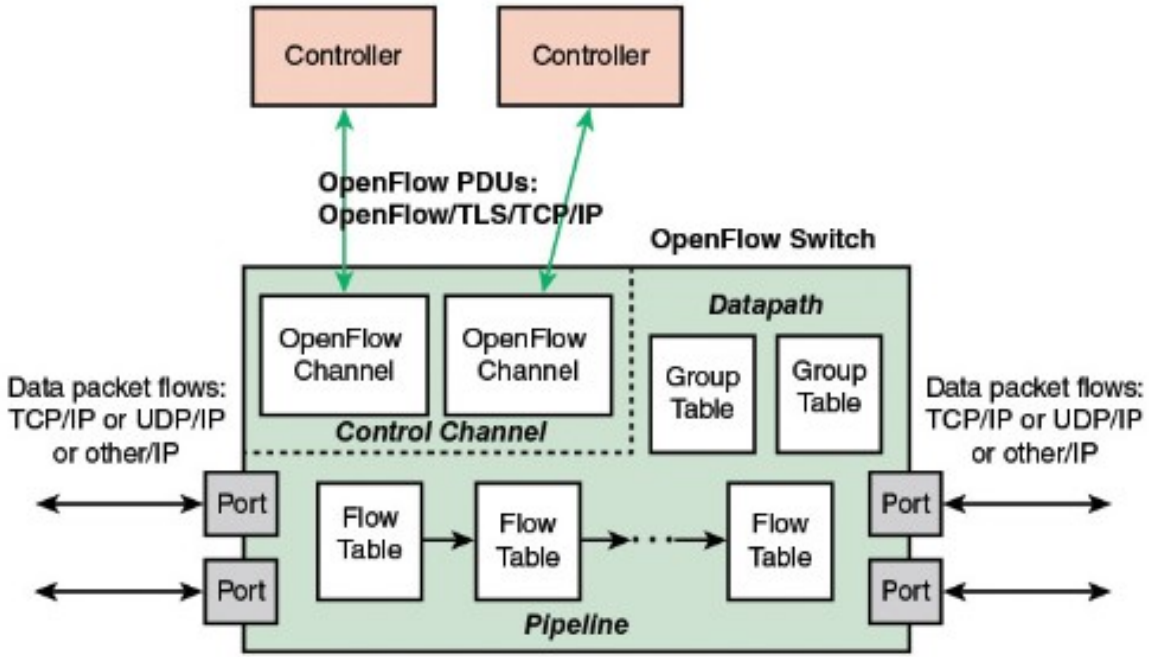
OpenFlow Basics

- Core protocol for SDN
- Flow tables in switches
- Controller instructs switches on packet forwarding
- Match-action rules: match packet → apply action

OpenFlow

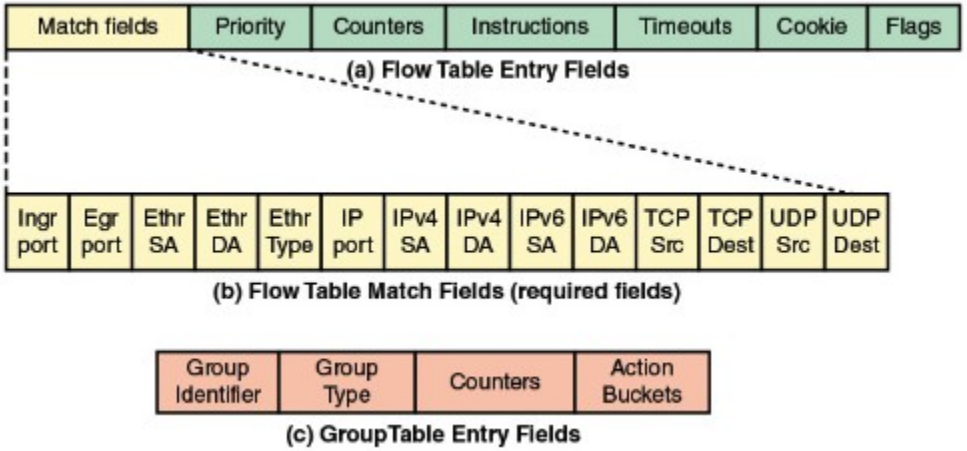

SDN Controllers
running OpenFlow
OpenFlow
Switches
End systems
OpenFlow Protocol over TLS
Data flows

OpenFlow switch

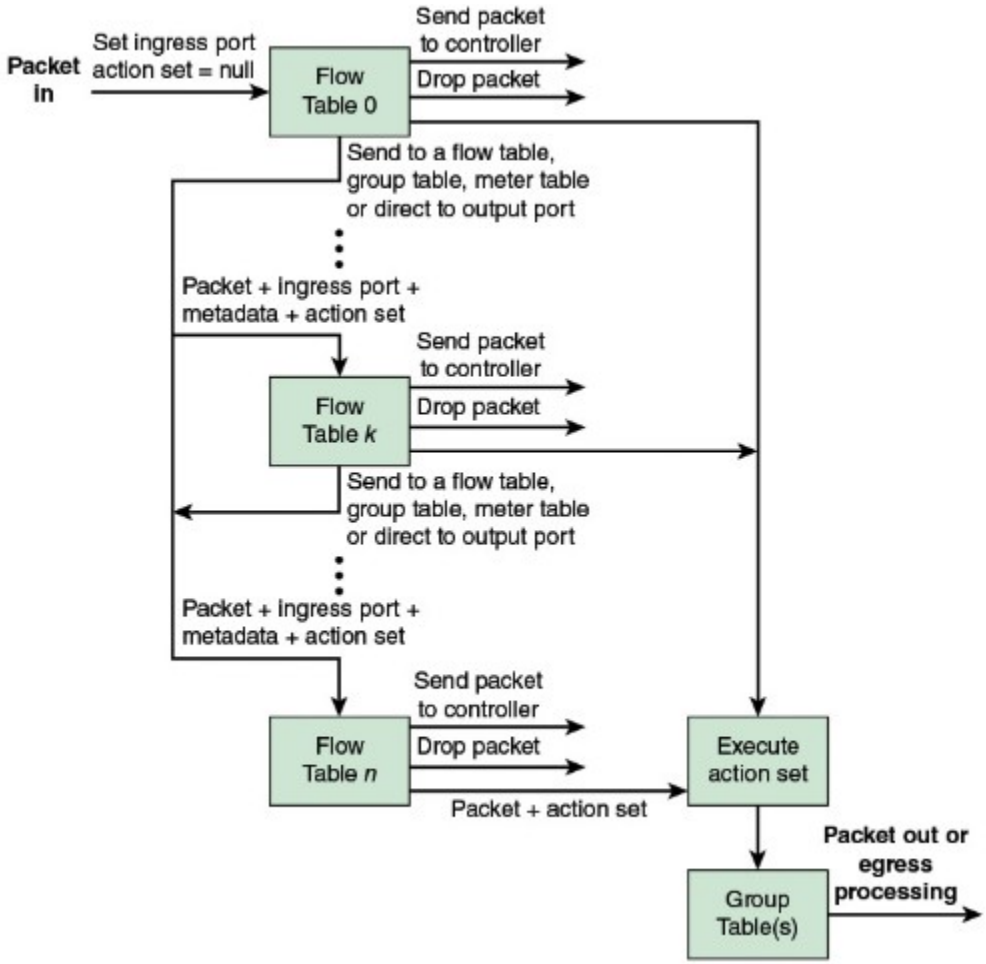


OpenFlow – FlowTable structure

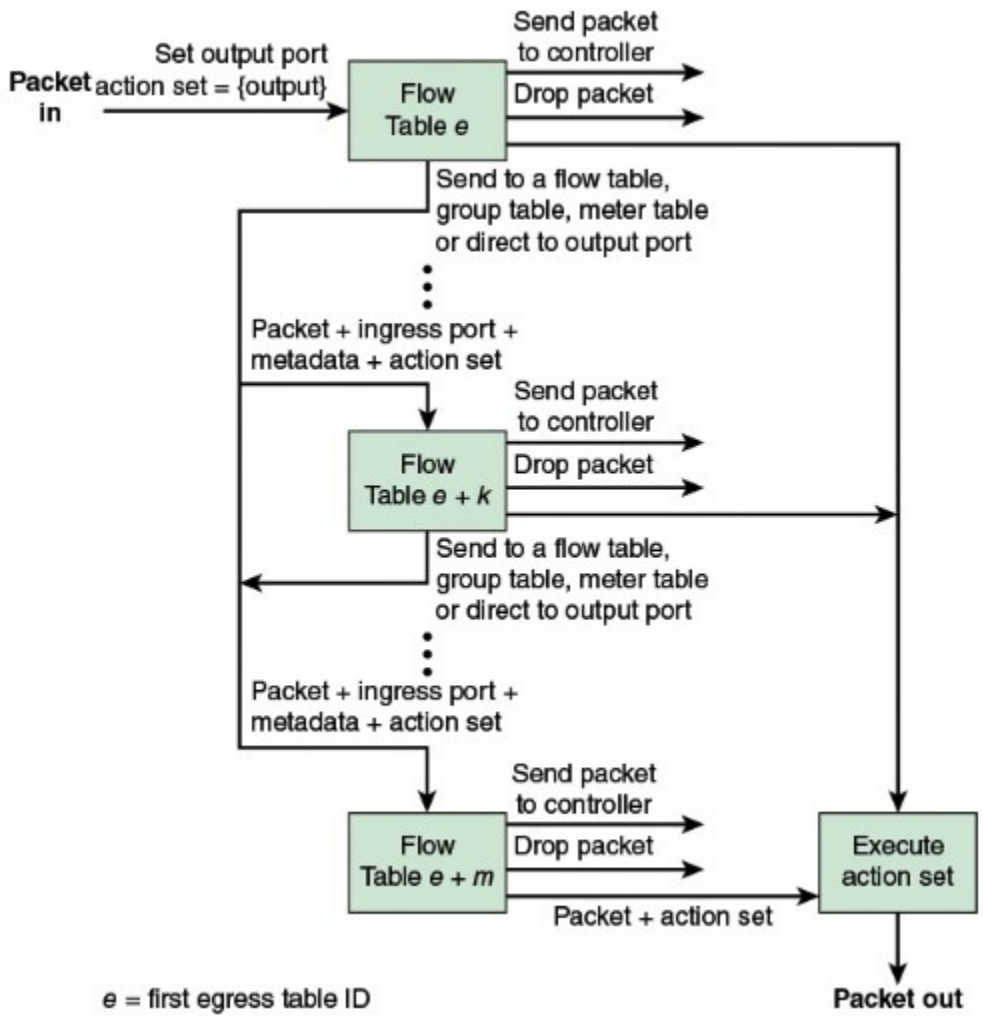
- Each packet entering the switch passes through one or more flow tables
- Each packet is matched to a flowtable entry and sent to an egress port or to another flow table



OpenFlow – Ingress processing of a packet



OpenFlow – Egress processing of a packet



Programmable Data Plane

- P4 language for custom packet processing
- Programmable switch pipelines
- Use cases: load balancing, custom packet headers, telemetry

SDN Virtualization

- Overlay networks: VXLAN, GRE, Geneve
- NFV vs SDN – complementary concepts
- Virtual switches: Open vSwitch, VPP

SDN Controllers

- Centralized intelligence for network automation
- Features: topology discovery, flow rule distribution, traffic engineering
- Popular controllers: OpenDaylight, ONOS, Ryu, Floodlight

SDN Use Cases: Data Centers

- Multi-tenant networks with isolation
- Automated provisioning and scaling
- East-west traffic optimization

SDN Use Cases: Service Providers

- 5G network slicing
- WAN SDN for traffic optimization
- Carrier-grade traffic engineering

SDN Use Cases: Enterprise & Campus

- Centralized policy and network segmentation
- Dynamic QoS for applications
- Identity-based access control and security automation

SDN Tools & Emulation

- **Mininet** – virtual networks for labs
- **GNS3 / EVE-NG** – industry-grade emulation
- **P4 Development Tools** – BMv2 switch, Tofino, P4Runtime

SDN Challenges

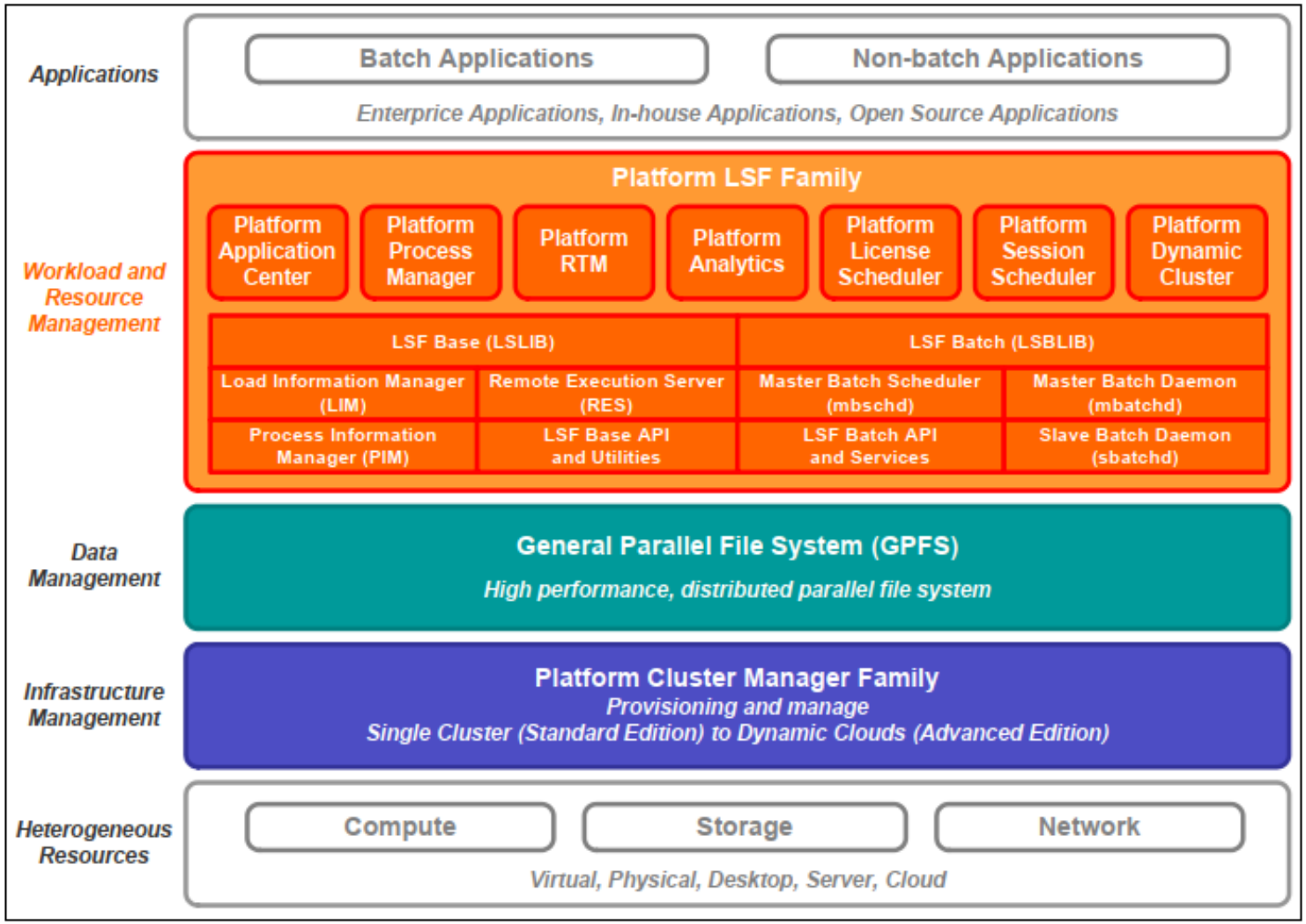
- Controller scalability and consistency
- Reliability and single point of failure
- Security: controller attacks, API misuse
- Standardization issues: multiple southbound protocols

Operating Systems for Computer Clusters

Functions of the OS of a HPC Cluster

- Provisioning computing nodes (installing/updating OS on computing nodes) (xCAT)
- Management of the computing resources of the cluster (PCM)
- Workload management (job/task scheduling) (Platform LSF)
- Distributed file system (GPFS, NFS)
- Hardware management and system monitoring (PCM)
- Scalability (PCM)

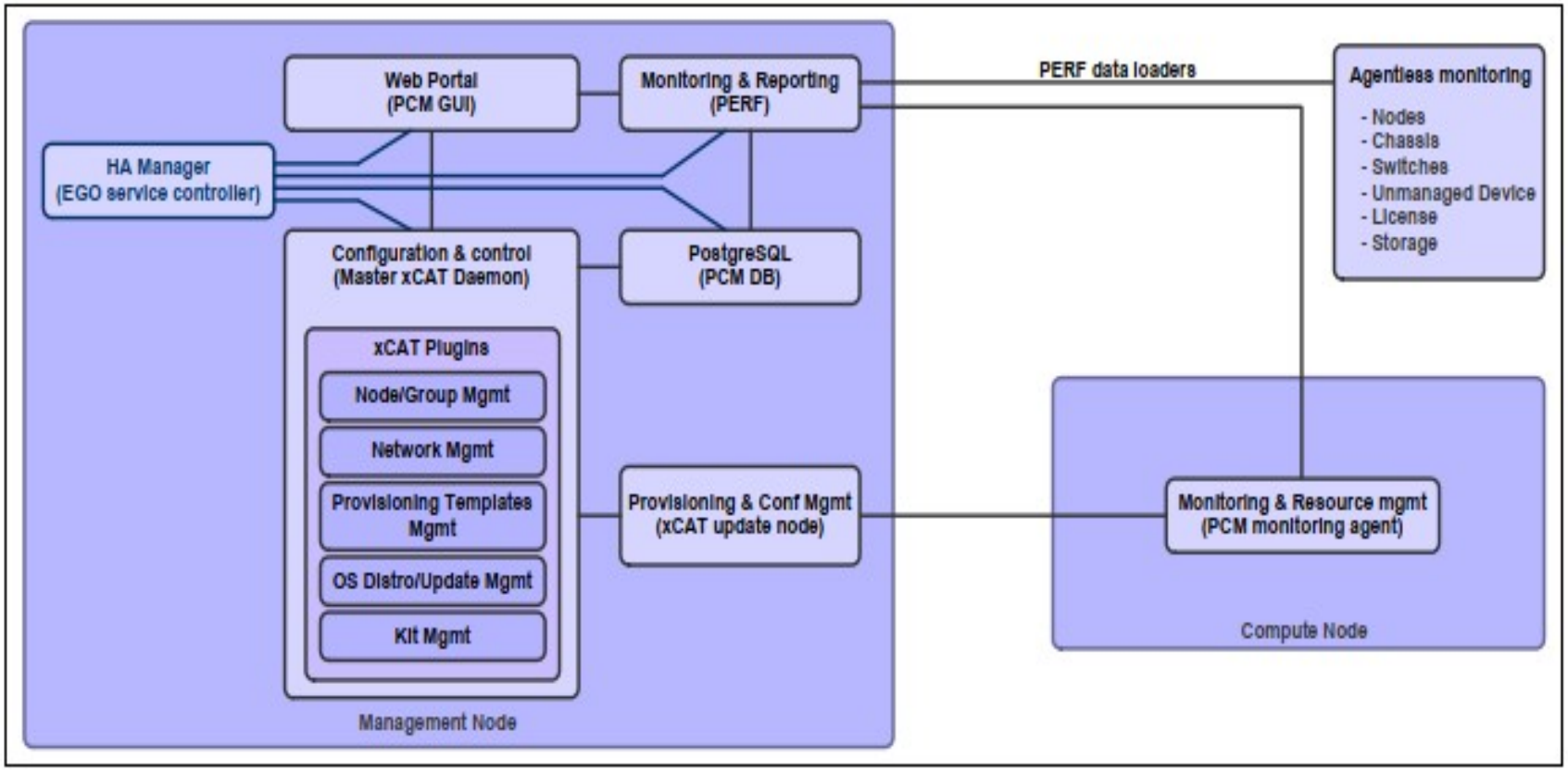
HPC Cluster layered architecture



IBM PCM – Platform Cluster Manager

- Manages the resources of the cluster (using xCAT as provisioning engine)
- Provides system monitoring and reporting
- Has centralized web portal
- Provides remote control of compute nodes

IBM PCM component architecture

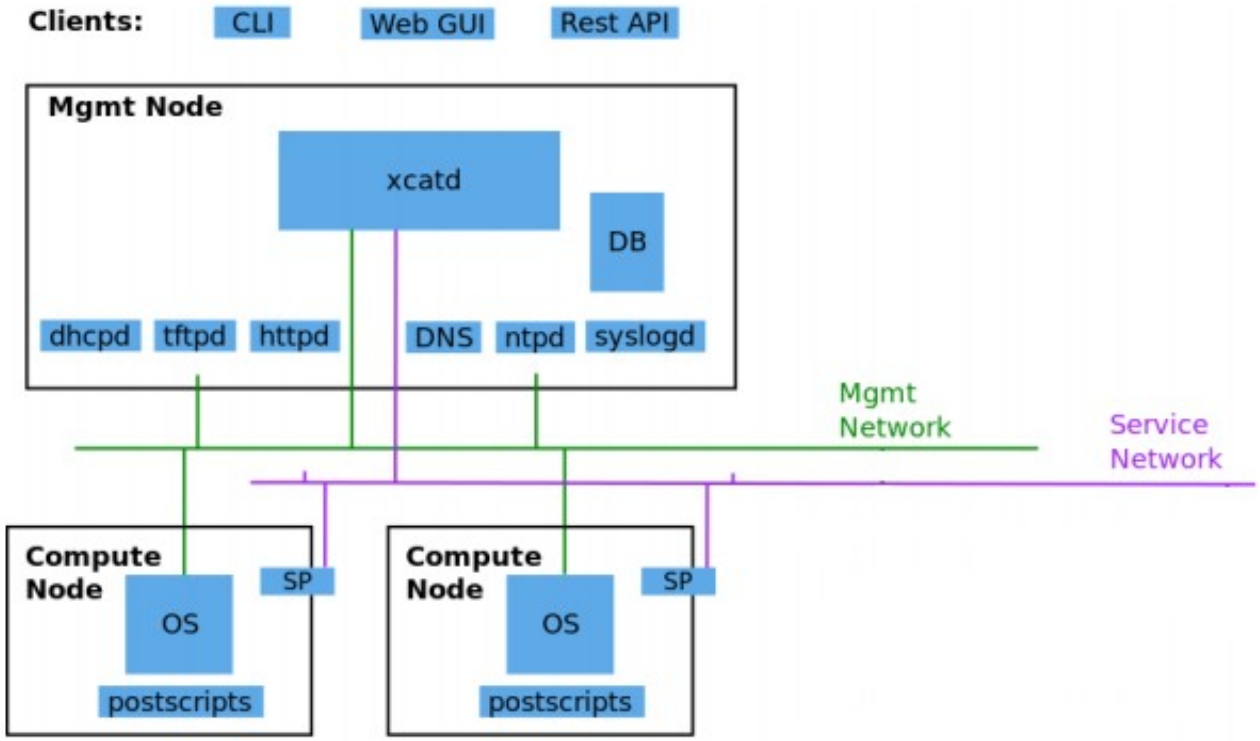


XCAT – EXTREME CLOUD ADMINISTRATION TOOLKIT

xCAT usages

- Discover the hardware servers
- Execute remote system management
- Provision operating systems on physical or virtual machines
- Provision machines in Diskful (stateful) and Diskless (stateless)
- Install and configure user applications

xCAT architecture



xCAT architecture (cont.)

- Mgmt node: management node, server where XCAT is installed and which is used as a single point in managing the cluster; nodes are defined in postscript database
- Compute node: a compute node from the clusted which is managed by xCAT
- Network services (dhcpd, tftpd, httpd, dns, ntpd, syslogd) : used for managing the cluster nodes, automatically configured
- Service Processor (SP): embedded system in the server hardware, used to perform out-of-band hardware control (IMM –Integrated Management Module for IBM, iLo for HP, iDRAC for DELL)
- Management network: used to provision/install compute nodes
- Service network: used by the management node to control the SP on the nodes

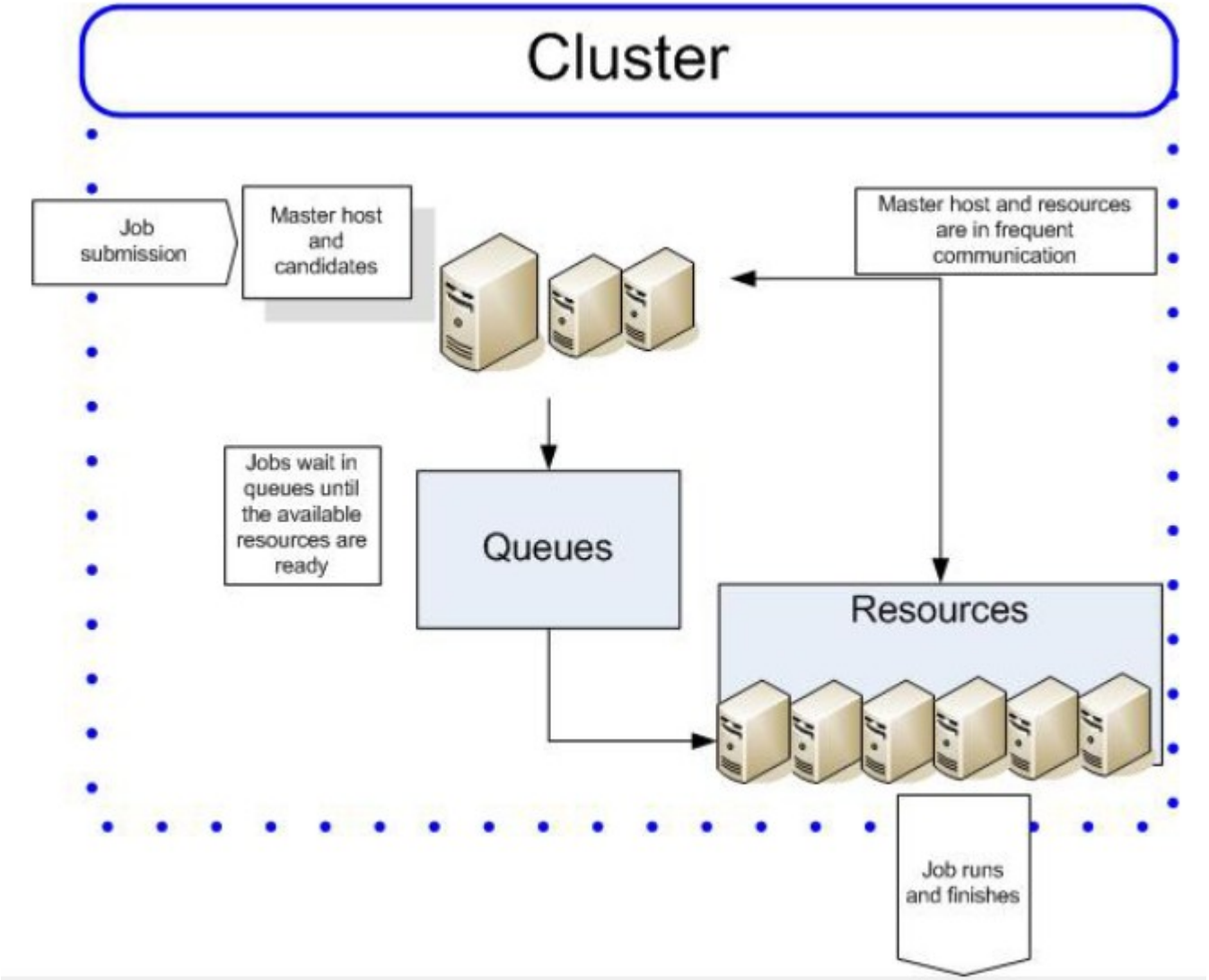
Start/stop the xcatd service

```
service  xcatd  { stop | start | restart | status
                                          }
```

Platform LSF

- LSF = Load Sharing Facility
- is job scheduling engine
- manages hardware resources and job requirements and finds the best execution plan
- monitors job execution progress
- has multiple job queues with several scheduling policies in the same cluster: FCFS (First Come First Served), Service Level Agreement (SLA) scheduling, Fairshare scheduling
- allows resource reservation
- allows resizable jobs

Job Processing with LSF



LSF Architecture

- LSF Base: provides load sharing services (resource usage information, process information host selection, remote execution)
  - Load Information Manager (LIM): LIM on a host monitors the host's load and reports it to the LIM running on headnode which provides the info to apps.
  - Process Information Manager(PIM): runs on each host, collects info about job processes and reports them to *sbatchd*
  - Remote Execution Server (RES): runs on each hosts and executes tasks on that host
- LSF Batch: provides job execution with load balancing and policy driven resource usage
  - *mbschd* (Master Batch Scheduler daemon): runs on headnode, receives job submissions, maintains queues, job status, dispaches jobs on compute nodes for execution
  - *sbatchd* (Slave Batch daemon): runs on each host, receives jobs from *mbschd*, creates a child *sbatchd* for each job, gives the job to RES in order to execute it
- LSF Libraries: provides API for interfacing with LSF

LSF – view daemon parameters

- Show mbatchd conf. : *badmin showconf mbd*
- Show sbatchd conf. on host: *badmin showconf sbd host*
- Show lim conf. on host: *lsadmin showconf lim*
- Summary*: badmin showstatus*

IBM GPFS - General Parallel File System

- shares NSDs (Network Storage Disks) between the hosts of a GPFS cluster
- the data access is done at block level (not at file level)
- a NSD can be created from a local hard disk drive, from several local hard disk drives (connected in RAID or as a multipath device) or from the partitions/LUNs (Logical Unit Number – an ID of a storage device in a SCSI environment)
- the NDSs are exposed to the hosts computers in the GPFS cluster by NSD servers.

UBB’s IBM Intelligent Cluster (2016)

- HPC Cluster + Cloud System
- Built by IBM (costed approx. 1 mil. EUR, without VAT)
- Consists of: 4x 42U computing racks, 4x cooling rack(+exterior unit), 2 UPS enclosures
- Performance: Rpeak 62 Tflops (theoretical), Rmax 40 Tflops

UBB’s IBM NextScale (HPC Cluster)

- 68 Nx360 M5 computing nodes
  - 50 nodes: 2x Intel Xeon E5-2660 v3 CPU, 10 cores; 128GB RAM; 2 HDD SATA 500 GB
  - 12 nodes with 2 Nvidia K40X GPU
  - 6 nodes with Intel Phi
- Fast networking: 56 Gb/s (Infiniband Mellanox FDR switch SX6512 with 216 ports, 1:1 subscription rate)
- Storage: IBM GPFS (General Parallel File System) NetApp E5660 Total: 72TB
- Backup: IBM TS3100 Tape library
- Mgmt. Software: IBM Platform HPC 4.2 and RedHat Linux Enterprise 7.2 for comp. nodes
- Others: 2 management nodes, 2 NSD, Fast Ethernet

UBB’s IBM Flex System (Cloud System)

- 10 virtualization servers Flex System x240
  - 128 Gb RAM / server
  - CPU 2 x Intel Xeon E5-2640 v2 / server
  - HDD 2 x SSD SATA 240 Gb / server
- 1 management server
- Cloud Software: IBM cloud manager with OpenStack 4.2
- Management and Monitor Software: IBM Flex System Manager software stack
- Virtualization Software: Vmware vSphere Enterprise 5.1

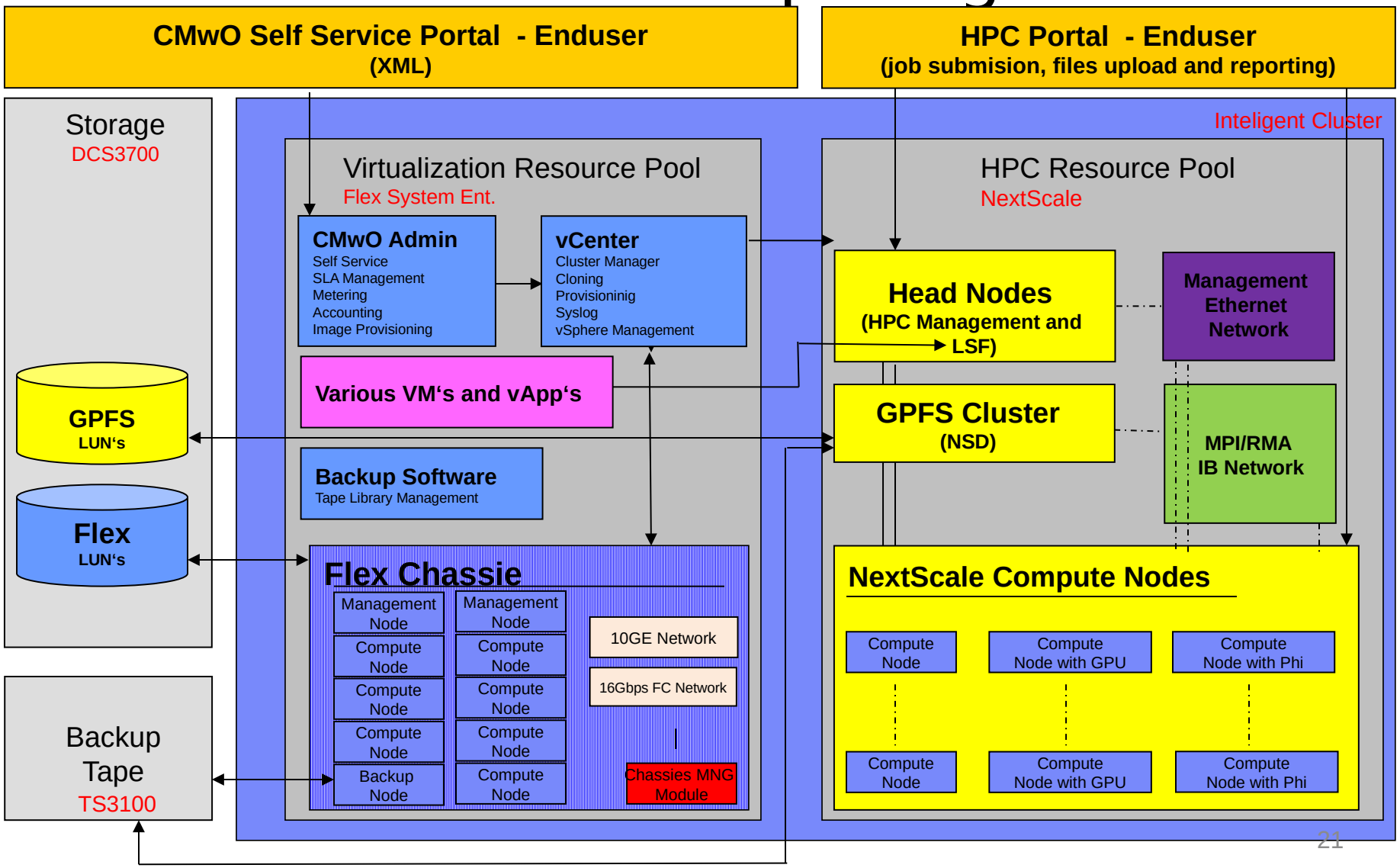
Cluster application software

- Inter Parallel Studio
- Matlab
- Mathematica
- Lumerical FDTD
- Ansys
- Comsol
- Multiphysics Gaussian
- Material Studio
- ...

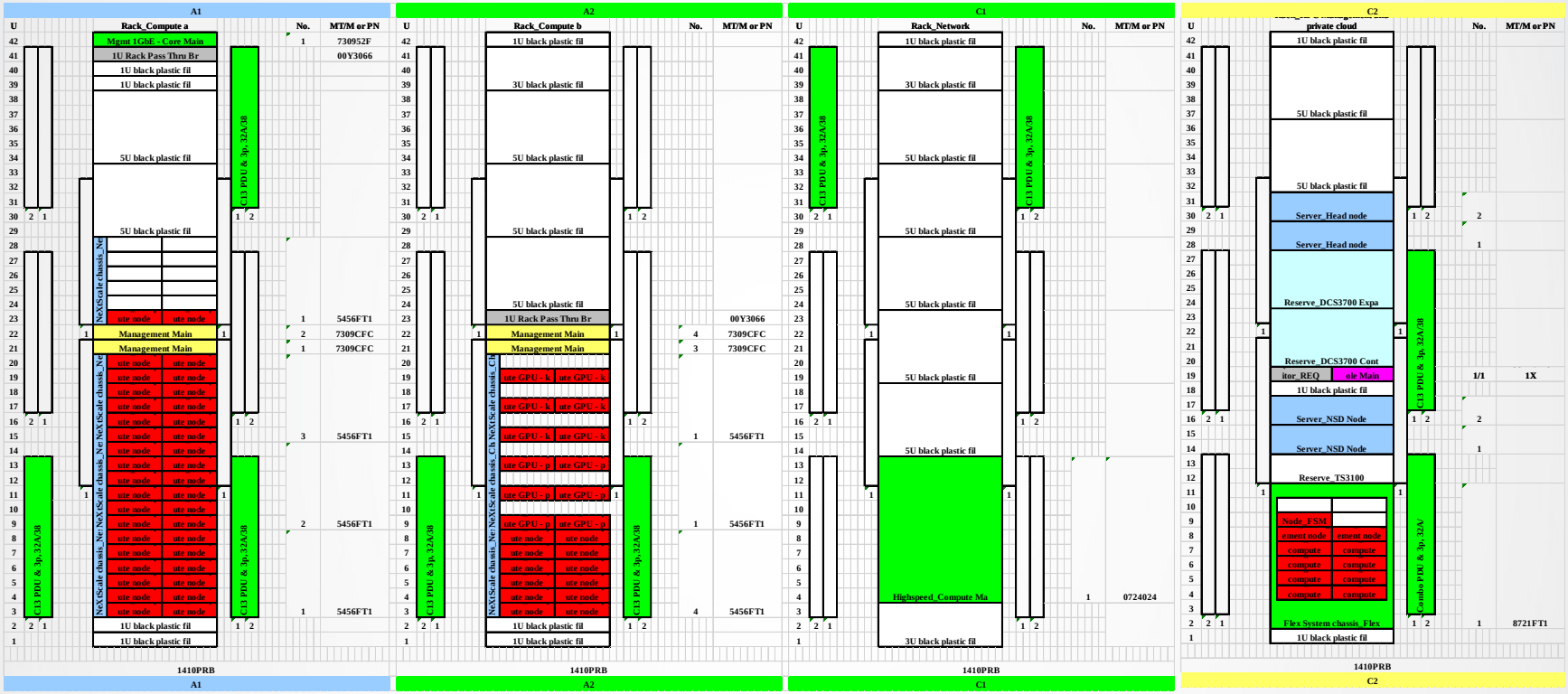
Cluster physical view

managed by **Faculty of Mathematics and Computer Science & Faculty of Economic Studies**

IT Architecture for HPC and Private Cloud Computing



IBM Intelligent Cluster - Rack and floor view



Rack Locations		
	Rack_HPC Management and private cloud	C2
	Rack_Compute b	A2
	Rack_Compute a	A1
	Rack_Network	C1
	Floor Diagram	
	1	2
A	Rack_Compute a	Rack_Compute b
B	cold	cold
C	Rack_Network	Rack_HPC Management and private cloud

© 2012 IBM Corporation